

**Федеральное государственное автономное образовательное учреждение
высшего образования «Уральский федеральный университет
имени первого Президента России Б. Н. Ельцина»**

На правах рукописи

БОРОДИН Андрей Михайлович

**МОДЕЛИ, АЛГОРИТМЫ И ПРОГРАММНЫЕ СРЕДСТВА
ОРГАНИЗАЦИИ ОБОБЩЁННЫХ ДЕРЕВЬЕВ ПОИСКА
ДЛЯ ДОСТУПА К МНОГОМЕРНЫМ ДАННЫМ В
СИСТЕМАХ УПРАВЛЕНИЯ БАЗАМИ ДАННЫХ**

Специальность 2.3.5 — Математическое и программное обеспечение
вычислительных систем, комплексов и компьютерных сетей

Диссертация на соискание учёной степени
доктора технических наук

Научный консультант:

Екатеринбург — 20__

ОГЛАВЛЕНИЕ

Введение	5
1 Состояние методов индексирования многомерных данных	12
1.1 Задача доступа к многомерным данным	12
1.2 Деревья пространственного индексирования	12
1.3 Обобщённое дерево поиска	13
1.4 Индексирование в основной памяти и битовые карты	14
1.5 Размерность данных и её следствия	14
1.6 Верификация индексных структур	15
1.7 Нерешённые задачи и постановка задач исследования	15
2 Модели эффективности доступа к многомерным данным	17
2.1 Обозначения	17
2.2 Ветвистость узла	17
2.3 Стоимость поиска и перекрытие ключей	18
2.4 Влияние размерности	19
2.5 Стоимость построения индекса	20
2.5.1 Построение последовательной вставкой	20
2.5.2 Построение сортировкой	20
2.6 Стоимость сопровождения	21
2.7 Выводы по главе	21
3 Алгоритмы повышения ветвистости и снижения перекрытия ключей	23
3.1 Избыточные представления ключа	23
3.1.1 Постановка	23
3.1.2 Решение	23
3.2 Покрывающие атрибуты	24
3.2.1 Постановка	24
3.2.2 Решение	24
3.3 Модификация элемента на странице	25

3.3.1	Постановка	25
3.3.2	Решение	25
3.4	Внутристраничное индексирование	26
3.4.1	Постановка	26
3.4.2	Измерение процессорной составляющей	26
3.4.3	Предлагаемый подход	27
3.4.4	Результаты и состояние работы	28
3.5	Функция штрафа и качество разбиения	29
3.6	Экспериментальная оценка	30
3.7	Выводы по главе	30
4	Построение обобщённого дерева поиска сортировкой	31
4.1	Постановка	31
4.2	Интерфейс поддержки сортировки	31
4.3	Порядок, сохраняющий локальность	32
4.4	Правые ссылки при построении снизу вверх	33
4.5	Перекрытие нелистовых ключей	33
4.5.1	Проблема	33
4.5.2	Алгоритм	34
4.6	Специализация компараторов	35
4.7	Границы применимости	35
4.8	Экспериментальная оценка	36
4.9	Выводы по главе	36
5	Конкурентное сопровождение индекса	37
5.1	Обход в физическом порядке	37
5.2	Удаление и переиспользование страниц	37
5.3	Условие безопасности переиспользования	38
5.4	Гранулярность блокировок при сборке мусора	39
5.4.1	Отмена второго изменения и её значение	40
5.5	Предикатные блокировки	42
5.6	Согласованность индекса при конкурентном создании	43
5.7	Упреждающее чтение	43
5.8	Экспериментальная оценка	44
5.8.1	Порядок обхода	44

5.8.2	Возврат страниц в оборот	46
5.8.3	Время ожидания блокировок в обратном индексе . . .	46
5.9	Выводы по главе	46
6	Верификация структурных инвариантов индекса	48
6.1	Постановка	48
6.2	Инварианты обобщённого дерева поиска	48
6.3	Проверка согласованности ключей родителя	49
6.4	Обход правых ссылок со сцеплением блокировок	50
6.5	Нормализация индексных кортежей	51
6.6	Проверка на реплике и в процессе восстановления	51
6.7	Коды ошибок повреждения данных	51
6.8	Экспериментальная оценка	52
6.9	Выводы по главе	52
7	Экспериментальная оценка и внедрение	53
7.1	Методика измерений	53
7.2	Наборы данных	54
7.3	Перекрытие ключей при разных стратегиях разбиения	54
7.4	Построение индекса	56
7.5	Поиск	56
7.6	Сопровождение	56
7.7	Верификация	56
7.8	Сопоставление с аналитическими моделями	56
7.9	Границы применимости	56
7.10	Внедрение	57
7.10.1	Основная ветка СУБД PostgreSQL	57
7.10.2	Промышленная эксплуатация	57
7.10.3	Учебный процесс	57
7.10.4	Предшествующая открытая реализация	58
7.11	Выводы по главе	58
	Заключение	59
A	Перечень результатов, принятых в основную ветку PostgreSQL	65

ВВЕДЕНИЕ

Актуальность темы исследования

Многомерные данные перестали быть принадлежностью специализированных аналитических систем. Геометрические и географические объекты, диапазоны значений и дат, полнотекстовые сигнатуры, векторные представления, многомерные кубы показателей — всё это сегодня хранится и обрабатывается универсальными системами управления базами данных (СУБД) наравне со скалярными типами. Соответственно изменилось и требование к методу доступа: индекс над многомерными данными должен быть не исследовательским прототипом, а штатным механизмом СУБД, который строится за приемлемое время, обслуживается конкурентно с пользовательской нагрузкой, восстанавливается после сбоя и поддается проверке на непротиворечивость.

Универсальным ответом на это требование стало обобщённое дерево поиска (generalized search tree, GiST), предложенное Дж. Хеллерстайном [1]. Обобщённое дерево выносит за скобки общую часть всех сбалансированных древовидных методов доступа — страничную организацию, расщепление узла, конкурентный поиск, журналирование — и оставляет разработчику класса операторов только предметную часть: как объединить ключи, как оценить штраф за расширение ключа, как разбить переполненный узел, как проверить, может ли поддерево содержать ответ. За счёт этого один и тот же код дерева обслуживает R-дерево [2], дерево диапазонов, дерево полнотекстовых сигнатур и произвольные пользовательские структуры.

Обобщённость, однако, имеет цену. Класс операторов ничего не сообщает дереву о том, существует ли для индексируемых данных порядок, сохраняющий локальность; поэтому индекс традиционно строился только последовательной вставкой. Ключ хранится в форме, о представлении которой ядро СУБД не знает ничего; поэтому в узле хранились избыточные служебные представления, снижавшие ветвистость. Сборка мусора обходила дерево в логическом порядке, порождая случайное чтение, и не возвращала освободившиеся страницы в оборот. Наконец, для универсальной структуры

сложнее сформулировать проверяемые инварианты, и до недавнего времени средств контроля целостности такого индекса в промышленных СУБД не было.

Каждое из перечисленных ограничений не является принципиальным свойством обобщённого дерева поиска — это следствие того, что развитие структуры исторически сосредоточилось на алгоритмах поиска и разбиения узла. Снятие этих ограничений и составляет содержание настоящей работы.

Степень разработанности темы

Теория пространственного индексирования оформилась в работах А. Гуттмана (R-дерево, [2]), Н. Бекманна и соавторов (R*-дерево [3], RR*-дерево [4]), где исследовались критерии качества разбиения узла: перекрытие ключей, покрытая площадь, периметр. Обобщение древовидных методов доступа выполнено Дж. Хеллерстайном [1]; конкурентность и восстановление обобщённого дерева исследованы М. Корнакером [5], откуда происходит схема с правыми ссылками и номерами последовательности узлов, восходящая к алгоритму Лемана и Яо для B-дерева [6]. Промышленная реализация обобщённого дерева поиска выполнена О. Бартуновым и Т. Сигаевым [7]. Предикатные блокировки для сериализуемой изоляции моментальных снимков описаны Д. Портсом и К. Гриттнером [8].

Применение пространственного индексирования к аналитическим данным и модели оценки его эффективности рассмотрены автором ранее [9, 10, 11, 12] и в кандидатской диссертации [13]; задача доступа к данным высокой размерности — в [14].

Вне рассмотрения этих работ остался жизненный цикл индекса целиком: построение индекса как самостоятельная задача (а не как повторная вставка), конкурентное сопровождение, возврат страниц в оборот и верификация структурных инвариантов. Именно эти вопросы определяют, применим ли метод доступа в промышленной эксплуатации, и именно они составляют предмет настоящей работы.

Объект и предмет исследования

Объект исследования: методы доступа к многомерным данным в системах управления базами данных.

Предмет исследования: модели и алгоритмы построения, поиска и сопровождения обобщённых деревьев поиска.

Цель и задачи

Цель работы: снижение вычислительной и введено-выводной стоимости построения, поиска и сопровождения индексов многомерных данных за счёт развития моделей и алгоритмов обобщённых деревьев поиска.

Для достижения цели решаются задачи:

1. Построить аналитические модели стоимости операций над обобщённым деревом поиска, связывающие ветвистость узла, перекрытие ключей и размерность данных с числом обращений к страницам.
2. Разработать алгоритмы повышения ветвистости узла и снижения перекрытия ключей.
3. Разработать алгоритм построения обобщённого дерева поиска сортировкой, обеспечивающий кратное ускорение построения без деградации качества дерева.
4. Разработать алгоритмы конкурентного сопровождения дерева: сборку мусора, удаление и переиспользование страниц, блокировки.
5. Разработать методы верификации структурных инвариантов индекса, пригодные для промышленной эксплуатации.
6. Реализовать разработанные алгоритмы в СУБД PostgreSQL и экспериментально оценить их эффективность и границы применимости.

Научная новизна

1. Впервые предложено сведение задачи построения *обобщённого* дерева поиска к сортировке: порядок, сохраняющий локальность, задаётся не ядром дерева, а классом операторов через интерфейс поддержки сортировки, что делает приём применимым к произвольному классу операторов, а не только к R-дереву.
2. Впервые выявлен и устранён основной побочный эффект сортированного построения — рост перекрытия нелистовых ключей, из-за которого выигрыш во времени построения оборачивается замедлением поиска.
3. Предложен алгоритм сборки мусора в обобщённом дереве поиска, выполняющий обход в физическом порядке страниц и возвращающий освободившиеся страницы в оборот; условие безопасности переиспользования сформулировано в терминах полных (64-разрядных) идентификаторов транзакций, что снимает известное ограничение, связанное с заикливанием 32-разрядных счётчиков.
4. Предложен метод верификации структурных инвариантов обобщённого дерева поиска и обратного индекса, работающий на действующей системе и на реплике, в том числе со сцеплением блокировок при обходе правых ссылок.
5. Развиты аналитические модели стоимости доступа к многомерным данным: в отличие от [10], модели связывают параметры страничной организации узла с наблюдаемой стоимостью запроса в промышленной СУБД.

Теоретическая и практическая значимость

Теоретическая значимость состоит в том, что задачи построения и сопровождения индекса поставлены и решены на уровне *класса* структур, а не отдельной структуры: результаты формулируются в терминах интерфейса класса операторов и переносятся на любую структуру, реализующую этот

интерфейс.

Практическая значимость определяется характером внедрения: все предложенные алгоритмы приняты в основную ветку СУБД PostgreSQL и поставляются в составе выпусков 10–19. Это отличает работу от исследований, останавливающихся на прототипе: результаты прошли независимое рецензирование сообществом разработчиков, эксплуатируются в промышленных инсталляциях, а эксперименты воспроизводимы на общедоступных выпусках. Каждый результат идентифицируется конкретным изменением в репозитории (приложение А). Часть результатов работы вошла в учебную и справочную литературу по устройству СУБД [15].

Методология и методы исследования

Использованы методы теории структур данных и алгоритмов, теории систем управления базами данных, теории вероятностей и математической статистики, методы экспериментального исследования производительности программных систем.

Помимо этого, в работе последовательно проводятся два принципа, вытекающие из характера предмета исследования.

Приоритет открытой реализации. Результатом считается не описание алгоритма, а его реализация, принятая в основную ветку открытой СУБД и доступная всем её пользователям. Публикация фиксирует и объясняет результат, но не заменяет его: закрытая реализация непроверяема третьей стороной, а следовательно, непроверяемо и утверждение о её свойствах. Этот принцип проводился автором и до настоящей работы: первая программная реализация методов доступа была зарегистрирована как открытый электронный ресурс [16] — при том, что распространения она не получила и практическое значение имела лишь как прецедент.

Эксплуатация как окончательная проверка. Утверждение о свойствах конкурентного алгоритма нельзя признать проверенным по результатам рецензирования публикации. Рецензент проверяет рассуждение при заявленных предпосылках; предпосылки же относятся не к алгоритму, а к остальной части системы, и их нарушение обнаруживается лишь длительной эксплуатацией на разнообразных нагрузках. В §5.4.1 разобран случай, когда опуб-

ликованный и отрецензированный алгоритм пришлось отменить именно по такому основанию. Из этого принципа вытекает и требование главы 6: свойства, не проверяемые функциональным тестом, должны проверяться инвариантами структуры.

Достоверность результатов обеспечивается согласованностью аналитических моделей с результатами измерений, независимым рецензированием всех программных реализаций в сообществе разработчиков PostgreSQL и их эксплуатацией в промышленных системах на протяжении нескольких выпусков.

Положения, выносимые на защиту

1. Аналитические модели стоимости построения, поиска и сопровождения обобщённого дерева поиска многомерных данных, связывающие ветвистость узла и перекрытие ключей с числом обращений к страницам.
2. Алгоритмы повышения ветвистости узла за счёт отказа от избыточных представлений ключа и поддержки покрывающих атрибутов, а также алгоритм внутристраничного индексирования, снимающий линейную по ветвистости составляющую стоимости просмотра узла.
3. Алгоритм построения обобщённого дерева поиска сортировкой по порядку, сохраняющему локальность, с компенсацией роста перекрытия нелистовых ключей.
4. Алгоритмы конкурентного сопровождения обобщённого дерева поиска: обход в физическом порядке, удаление и безопасное переиспользование страниц на основе полных идентификаторов транзакций, предикатные блокировки.
5. Метод верификации структурных инвариантов индексных структур, применимый в промышленной эксплуатации, в том числе на реплике.
6. Программная реализация перечисленных алгоритмов в СУБД PostgreSQL и результаты экспериментальной оценки их эффективности и границ применимости.

Степень достоверности и апробация результатов

Результаты докладывались на международных конференциях BDAS (2015, 2016, 2017), IEEE ICBDA (2017), Dynamics (2016), USBEREIT (2018, 2019), а также на конференциях разработчиков PostgreSQL:

- PGConf.Russia, 2017 — «Возможности ускорения GiST: патчи, хаки, твики» (материал главы 3);
- PGCon, 2020 — «DIY: database index» (вынос метода доступа из ядра СУБД в расширение; главы 1, 3);
- PGCon, 2022 — «CREATE INDEX CONCURRENTLY implementation details» (глава 5);
- PGCon, 2023 — «GiST news and prospects» (главы 3–5);
- PGConf.Russia, 2026 — «Заметки о целостности данных» (глава 6).

Программные реализации прошли рецензирование в открытом процессе разработки PostgreSQL и включены в выпуски 10–19.

Публикации

По теме диссертации опубликовано __ работ, в том числе __ в изданиях, индексируемых Scopus и Web of Science, __ в изданиях из Перечня ВАК, а также монография [12]. Получено __ свидетельств о государственной регистрации программ для ЭВМ.

Структура и объём работы

Диссертация состоит из введения, семи глав, заключения, списка использованных источников и приложений.

1 СОСТОЯНИЕ МЕТОДОВ ИНДЕКСИРОВАНИЯ МНОГОМЕРНЫХ ДАННЫХ

1.1 Задача доступа к многомерным данным

Пусть данные организованы в D измерений $H_1, \dots, H_D$, каждое из которых представляет собой иерархию элементов. Записи адресуются набором $K = \{h_1, \dots, h_D\}$, $h_i \in H_i$. Аналитический запрос $Q\{h\}$ отбирает все записи, ключи которых подчинены заданным элементам иерархий, и агрегирует их.

В [13] показано, что при традиционном для СУБД индексировании по составному ключу число просматриваемых диапазонов растёт экспоненциально с D , и предложено отображение иерархии на числовую ось: каждому элементу h_i ставится в соответствие отрезок $[a_i, b_i]$ так, что отрезок потомка вложен в отрезок предка, а отрезки несвязанных элементов не пересекаются. При таком отображении аналитический запрос переходит в пространственный запрос типа «все точки внутри D -мерного параллелепипеда», а задача сводится к пространственному индексированию.

Дальнейшее изложение опирается на это сведение, но не ограничивается им: рассматриваемые структуры индексируют произвольные многомерные объекты, для которых определены операции объединения ключей и проверки пересечения.

1.2 Деревья пространственного индексирования

R-дерево [2] представляет собой сбалансированное дерево, узел которого содержит набор ограничивающих прямоугольников (ключей) вместе со ссылками на поддеревья. Ключ узла покрывает все ключи его потомков. Поиск спускается по всем поддеревьям, ключи которых пересекаются с запросом; в отличие от B-дерева, путей спуска может быть несколько, и стоимость поиска определяется не высотой дерева, а числом узлов, ключи которых пересекаются с запросом.

Отсюда два критерия качества дерева:

- *перекрытие* ключей узлов одного уровня — чем больше суммарная площадь пересечений, тем больше поддеревьев приходится просматривать при точечном запросе;
- *покрытая площадь* — чем больше «пустого» пространства охватывает ключ, тем чаще запрос спускается в поддерево, не содержащее ответа.

R*-дерево [3] улучшает оба показателя за счёт учёта периметра при разбиении узла и принудительного перевставления части элементов. RR*-дерево [4] уточняет критерий выбора поддерева при вставке. Общее для всех вариантов: качество дерева определяется двумя решениями — выбором поддерева при вставке (функция штрафа) и разбиением переполненного узла.

1.3 Обобщённое дерево поиска

Обобщённое дерево поиска [1] абстрагирует описанную схему. Ядро дерева реализует страничную организацию, спуск, вставку, расщепление, конкурентный доступ и журналирование; предметная часть выносится в *класс операторов*, который для PostgreSQL состоит из функций:

- *consistent* — может ли поддерево с данным ключом содержать ответ на запрос;
- *union* — ключ, покрывающий заданный набор ключей;
- *penalty* — штраф за расширение ключа при вставке;
- *picksplit* — разбиение переполненного узла на две части;
- *same* — совпадение ключей;
- *compress, decompress* — преобразование значения во внутреннее представление и обратно;
- *fetch* — восстановление исходного значения из ключа, необходимое для индексного сканирования без обращения к таблице;
- *distance* — расстояние до запроса для поиска ближайших соседей.

Функции `consistent`, `union`, `penalty` и `picksplit` задают структуру дерева полностью: подставив в них прямоугольники, получаем R-дерево; подставив сигнатуры — дерево полнотекстового поиска; подставив диапазоны — дерево диапазонов.

Подробное описание реализации методов доступа в PostgreSQL, включая обобщённое дерево поиска, дерево с разбиением пространства и обратный индекс, приведено в [15]; там же изложены механизмы, с которыми методы доступа взаимодействуют, — буферный кэш, журнал предзаписи, блокировки и очистка.

В реализации PostgreSQL узел дерева отождествляется со страницей буферного кэша, что позволяет работать с данными, превышающими объём оперативной памяти. Алгоритмы спуска и расщепления реализованы без рекурсии по стеку. Конкурентный доступ организован по схеме Корнакера [5]: страницы связаны правыми ссылками, а расщепление узла оформлено так, что поиск, пришедший на расщеплённую страницу, может дойти до нужных данных по правой ссылке, не удерживая блокировку родителя. Соответствие ключа родителя ключам потомков восстанавливается отложено.

1.4 Индексирование в основной памяти и битовые карты

Для аналитических нагрузок с малым числом различных значений в измерении эффективны битовые карты; для резидентных в памяти наборов — структуры, оптимизированные под отсутствие страничного обмена. Сравнительный анализ этих подходов и деревьев выполнен в [11] и [12]. Общий вывод: битовые карты выигрывают при низкой мощности измерений и статических данных, деревья — при высокой мощности и обновляемых данных. Настоящая работа рассматривает второй случай.

1.5 Размерность данных и её следствия

С ростом D доля объёма D -мерного шара в описанном кубе стремится к нулю, а расстояния между случайными точками концентрируются около

среднего. Практическое следствие для древовидных индексов: ключи узлов начинают перекрываться почти полностью, и поиск вырождается в просмотр всего дерева. Границы применимости пространственного индексирования по размерности исследованы в [14]; там же рассмотрены подходы, снижающие эффективную размерность.

1.6 Верификация индексных структур

Ошибка в реализации метода доступа проявляется не отказом, а молчаливой потерей данных: запрос перестаёт находить существующие записи. В [17] описан подход к отладке индексных структур, основанный на явной проверке инвариантов. Однако средств такой проверки в составе промышленных СУБД для обобщённого дерева поиска и обратного индекса до недавнего времени не существовало.

Практическая сторона вопроса — какие ошибки эксплуатации и проектирования приводят к деградации и повреждению индексов — систематизирована в [18]; эта работа полезна как источник сведений о том, какие именно отклонения встречаются в промышленных инсталляциях, и потому задаёт требования к средствам контроля целостности.

1.7 Нерешённые задачи и постановка задач исследования

Обзор показывает, что литература сосредоточена на двух решениях, определяющих качество дерева, — выборе поддеревя и разбиении узла. За её пределами остаются:

1. *Ветвистость узла.* Она принимается за характеристику структуры, хотя определяется представлением ключа и наличием в узле служебных данных, и потому поддаётся оптимизации (глава 3).
2. *Построение индекса.* Рассматривается как многократная вставка, хотя при построении весь набор данных известен заранее (глава 4).

3. *Сопровождение*. Сборка мусора, возврат страниц в оборот и конкурентность обсуждаются для В-дерева, но не для обобщённого дерева поиска (глава 5).
4. *Верификация*. Инварианты структуры не формализованы и не проверяются в эксплуатации (глава 6).

Этим определяется постановка задач, сформулированных во введении. Прежде чем переходить к алгоритмам, в главе 2 строятся модели, показывающие, на какие именно параметры следует воздействовать.

2 МОДЕЛИ ЭФФЕКТИВНОСТИ ДОСТУПА К МНОГОМЕРНЫМ ДАННЫМ

Задача главы — выделить параметры обобщённого дерева поиска, воздействие на которые снижает стоимость операций, и оценить чувствительность стоимости к каждому из них. Стоимость всюду измеряется числом обращений к страницам, с разделением на последовательные и произвольные, поскольку именно это разделение определяет время выполнения на реальных носителях.

2.1 Обозначения

N	число индексируемых записей
B	размер страницы, байт
h	размер служебного заголовка страницы, байт
k	средний размер ключа, байт
p	размер ссылки на страницу либо на запись таблицы, байт
f	ветвистость узла — число элементов на странице
H	высота дерева
s	селективность запроса, доля отобранных записей
D	размерность данных
c_s	стоимость последовательного чтения страницы
c_r	стоимость произвольного чтения страницы, $c_r \gg c_s$

2.2 Ветвистость узла

Ветвистость определяется размером страницы и размером хранимого элемента:

$$f = \left\lfloor \frac{B - h}{k + p + \sigma} \right\rfloor, \quad (2.1)$$

где σ — размер служебных данных, сопровождающих элемент в узле. Слагаемое σ вводится явно, поскольку в реализации обобщённого дерева по-

иска оно не равно нулю: ключ может храниться в преобразованном виде, а вместе с ним — дополнительные атрибуты. Именно на σ и на k воздействуют алгоритмы главы 3.

Высота дерева и число листовых страниц:

$$H = \left\lceil \log_f \frac{N}{f} \right\rceil + 1, \quad L = \left\lceil \frac{N}{f\varphi} \right\rceil, \quad (2.2)$$

где $\varphi \in (0, 1]$ — коэффициент заполнения страницы.

Из (2.2) видно, что ветвистость влияет на высоту логарифмически, а на число страниц — обратно пропорционально. Поэтому эффект от роста f проявляется не столько в сокращении длины пути спуска, сколько в сокращении объёма индекса и объёма чтения при операциях, обходящих индекс целиком: построении, сборке мусора, верификации, сканировании по неселективному условию. Это наблюдение существенно: оптимизация ветвистости обычно мотивируется ускорением поиска, тогда как основной выигрыш она даёт на массовых операциях.

2.3 Стоимость поиска и перекрытие ключей

В отличие от В-дерева, поиск в обобщённом дереве спускается по всем поддеревьям, ключи которых удовлетворяют условию *consistent*. Введём коэффициент перекрытия уровня l

$$\omega_l = \mathbb{E} [|\{ u \in U_l : \text{consistent}(\text{key}(u), Q) \}| |], \quad (2.3)$$

то есть математическое ожидание числа узлов уровня l , ключи которых пересекаются со случайным точечным запросом Q . Для дерева без перекрытия $\omega_l = 1$ на всех уровнях. Тогда стоимость точечного запроса

$$P_{\text{point}} = \sum_{l=0}^{H-1} \omega_l c_r, \quad (2.4)$$

Существенно, что ω_l не является произведением перекрытий вышележащих уровней. Прямое измерение (§7.3) показывает, что для каждой стратегии разбиения величина ω_l выходит на характерное установившееся значение ω_∞ уже на первом-втором уровне под корнем и далее с глубиной не растёт.

Верхние уровни составляют исключение лишь потому, что там ω_l ограничено числом узлов уровня. Поэтому

$$P_{\text{point}} \approx \omega_{\infty} H c_r, \quad (2.5)$$

и качество разбиения входит в стоимость поиска *постоянным множителем на каждом шаге спуска*, а не накапливающимся по уровням. Практическое следствие то же по знаку, но иное по величине: ухудшение разбиения удорожает поиск в $\omega_{\infty}^{\text{плохое}}/\omega_{\infty}^{\text{хорошее}}$ раз независимо от объёма данных.

Для запроса селективности s к (2.4) добавляется просмотр листовых страниц, содержащих ответ:

$$P_{\text{range}} \approx \sum_{l=0}^{H-2} \omega_l c_r + sL c_r. \quad (2.6)$$

Отсюда первое следствие, определяющее содержание главы 4: любое ускорение построения индекса, увеличивающее ω_l хотя бы на верхних уровнях, оплачивается замедлением *всех* последующих запросов. Сравнение методов построения поэтому некорректно проводить по одному лишь времени построения.

2.4 Влияние размерности

При росте D ключи узлов стремятся покрыть весь диапазон по каждому измерению, и $\omega_l \rightarrow |U_l|$: поиск вырождается в полный просмотр уровня. Пусть данные равномерно распределены в единичном D -мерном кубе, а ключ узла — ограничивающий параллелепипед со стороной a по каждому измерению. Вероятность того, что случайный точечный запрос попадёт в ключ, равна a^D , и

$$\omega_l \approx |U_l| a_l^D, \quad a_l \approx \left(\frac{f^{l+1}}{N} \right)^{1/D}, \quad (2.7)$$

откуда $\omega_l \approx |U_l| f^{l+1}/N \approx 1$ для идеального разбиения. Практический смысл (2.7) в другом: показатель $1/D$ означает, что при больших D сторона ключа a_l близка к единице уже на средних уровнях, и малейшее отклонение разбиения от идеального даёт кратный рост ω_l . Именно поэтому качество раз-

биения критично для многомерных данных и некритично для одномерных. Экспериментальная граница применимости по D исследована в [14].

2.5 Стоимость построения индекса

2.5.1 Построение последовательной вставкой

Каждая вставка спускается от корня до листа, при необходимости расщепляя узлы. Спуск на каждом уровне требует произвольного чтения:

$$P_{\text{ins}} = N (H c_r + \gamma c_r), \quad (2.8)$$

где γ — среднее число дополнительных обращений на расщепление и обновление ключей родителей. Существенно, что обращения произвольные: порядок вставки не согласован с физическим расположением страниц, а рабочее множество верхних уровней помещается в кэш, тогда как листовых — нет.

2.5.2 Построение сортировкой

Если для класса операторов задан порядок, сохраняющий локальность, набор данных можно отсортировать и заполнять страницы подряд снизу вверх. Стоимость внешней сортировки в модели с блоком B и памятью M :

$$P_{\text{sort}} = \frac{2N}{B} \log_{M/B} \frac{N}{B} c_s + \frac{N}{f\varphi} c_s. \quad (2.9)$$

Сравнение (2.8) и (2.9) даёт два источника выигрыша: замена произвольного доступа последовательным ($c_r \rightarrow c_s$) и исключение расщеплений. Дополнительно возрастает φ : при заполнении подряд страница заполняется до заданного коэффициента, тогда как при построении расщеплениями $\varphi \approx 0,7$. Отсюда — и меньший объём индекса.

Ограничение метода очевидно из вывода: он требует порядка, сохраняющего локальность, которого класс операторов в общем случае не задаёт. Способ получить такой порядок и передать его ядру дерева рассматривается в главе 4.

2.6 Стоимость сопровождения

Сборка мусора обходит все страницы индекса. При обходе в логическом порядке (спуск по дереву) последовательность номеров страниц произвольна:

$$P_{\text{vac}}^{\text{log}} = (L + I) c_r, \quad (2.10)$$

где I — число внутренних страниц. При обходе в физическом порядке номеров страниц:

$$P_{\text{vac}}^{\text{phys}} = (L + I) c_s + \beta c_r, \quad (2.11)$$

где β — число страниц, которые пришлось перечитать из-за конкурентных вставок, переместивших ещё не обработанные элементы на уже пройденные страницы. Выигрыш составляет $(L + I)(c_r - c_s)$ и тем существеннее, чем больше отношение c_r/c_s — на магнитных дисках оно достигает двух порядков, на твердотельных накопителях сокращается, но остаётся значимым за счёт возможности упреждающего чтения: при известном заранее порядке страниц запросы к носителю выдаются пакетами, а не по одному.

Отдельная составляющая стоимости сопровождения — удержание блокировок. Если сборка мусора блокирует поддерево целиком на время обработки, стоимость для пользовательской нагрузки определяется не числом обращений к страницам, а временем ожидания; модель этой составляющей и способ его сокращения рассматриваются в §5.4.

2.7 Выводы по главе

1. Стоимость поиска определяется перекрытием ключей (2.4). Оно не накапливается с глубиной: каждая стратегия разбиения выходит на характерное установившееся значение ω_∞ , и стоимость составляет $\omega_\infty H$ (2.5). При большой размерности данных чувствительность к качеству разбиения резко возрастает (2.7).
2. Ветвистость (2.1) влияет на стоимость поиска логарифмически, а на объём индекса и стоимость массовых операций — обратно пропорци-

онально; воздействовать на неё можно через размер ключа k и служебные данные σ .

3. Построение сортировкой асимптотически выгоднее построения вставкой за счёт замены произвольного доступа последовательным и отказа от расщеплений, но допустимо только при контроле перекрытия нелистовых ключей.
4. Обход в физическом порядке при сопровождении даёт выигрыш, пропорциональный $c_r - c_s$ и объёму индекса.

Три параметра, поддающиеся воздействию без изменения класса операторов — ветвистость, перекрытие и порядок обращения к страницам, — определяют содержание глав 3–5.

3 АЛГОРИТМЫ ПОВЫШЕНИЯ ВЕТВИСТОСТИ И СНИЖЕНИЯ ПЕРЕКРЫТИЯ КЛЮЧЕЙ

Согласно (2.1), ветвистость узла определяется размером ключа k и размером сопутствующих служебных данных σ . В этой главе рассматриваются три источника избыточности, каждый из которых увеличивает k или σ без пользы для поиска, и алгоритмы их устранения; затем — способ снять зависимость стоимости просмотра узла от его ветвистости.

3.1 Избыточные представления ключа

3.1.1 Постановка

Интерфейс класса операторов обобщённого дерева поиска исторически требовал двух парных функций: `compress` преобразует индексируемое значение во внутреннее представление ключа, `decompress` — обратно. Замысел состоял в том, чтобы хранить в дереве сжатую форму значения: например, для многоугольника — его ограничивающий прямоугольник.

Однако для значительной части классов операторов преобразование не требуется: внутреннее представление совпадает с внешним. Реализация вынуждала и в этом случае определять функции, выполняющие тождественное преобразование. Помимо избыточного кода это имело два следствия:

1. ядро дерева не знало, совпадает ли хранимое представление с исходным значением, и потому не могло выполнить индексное сканирование без обращения к таблице, если класс операторов не определял ещё и функцию `fetch`;
2. тождественная функция `decompress` вызывалась для каждого элемента каждой просмотренной страницы, то есть $O(f)$ раз на узел.

3.1.2 Решение

Предложено сделать обе функции необязательными и придать их отсутствию семантику. Если `compress` не определена, ядро дерева знает, что

хранимое представление тождественно исходному значению, и:

- выполняет индексное сканирование без обращения к таблице, не требуя функции `fetch`;
- не вызывает преобразование при просмотре узла.

Реализация: d3a4f89d8a3 «Allow no-op GiST support functions to be omitted» (выпуск 11). Помимо снижения накладных расходов результат сокращает объём обязательного кода класса операторов — это существенно для расширяемости, поскольку порог входа для разработчика нового метода доступа определяется именно объёмом обязательной обвязки.

3.2 Покрывающие атрибуты

3.2.1 Постановка

Индексное сканирование без обращения к таблице возможно, если все запрашиваемые столбцы присутствуют в индексе. Наивный способ добиться этого — включить дополнительные столбцы в состав ключа. Для обобщённого дерева поиска он неприменим по двум причинам: во-первых, для типа дополнительного столбца может не существовать класса операторов; во-вторых, включение столбца в ключ увеличивает k на *всех* уровнях дерева, а согласно (2.1) и (2.2) это снижает ветвистость внутренних узлов и увеличивает высоту дерева.

3.2.2 Решение

Предложено разделить атрибуты индекса на ключевые и покрывающие. Покрывающие атрибуты хранятся только в листовых страницах, не участвуют в навигации, не входят в ключи внутренних узлов и не требуют класса операторов для своего типа. Формально: ветвистость внутренних уровней определяется (2.1) с k по ключевым атрибутам, ветвистость листового уровня — с $k + k_{\text{inc}}$; поскольку по (2.2) высота зависит от ветвистости внутренних узлов, добавление покрывающих атрибутов её не увеличивает.

Существенное отличие от В-дерева: там покрывающие атрибуты введены преимущественно для того, чтобы наложить ограничение уникальности на часть атрибутов индекса, а здесь основной мотив — получить индексное сканирование для атрибутов, тип которых вообще не имеет класса операторов обобщённого дерева поиска.

Реализация: f2e403803fe «Support for INCLUDE attributes in GiST» (выпуск 12), для дерева с разбиением пространства — 09c1c6ab4bc «Support INCLUDE'd columns in SP-GiST» (выпуск 14).

3.3 Модификация элемента на странице

3.3.1 Постановка

Вставка в обобщённое дерево поиска сопровождается обновлением ключей на пути спуска: ключ родителя расширяется так, чтобы покрыть вставленный элемент. Каждое такое обновление — модификация элемента, уже находящегося на странице.

В исходной реализации модификация выполнялась как удаление элемента с последующим добавлением нового в конец страницы. Поскольку элементы на странице хранятся плотно, удаление вызывает перемещение всей области данных, лежащей «ниже» удаляемого элемента. Кроме того, недавно обновлённые элементы всегда оказывались в конце страницы, что разрушало соответствие между порядком указателей и физическим порядком данных и ухудшало локальность обращений к кэшу процессора.

3.3.2 Решение

Предложена операция перезаписи элемента на месте. Если размер нового элемента равен размеру старого — а при обновлении ключа это типичный случай, — данные на странице не перемещаются вовсе; при неравенстве размеров перемещается только разность. Порядок элементов сохраняется.

Реализация: b1328d78f88 «Invent PageIndexTupleOverwrite, and teach BRIN and GiST to use it» (выпуск 10). Результат опубликован в [19]; там же приведены измерения: сокращение времени вставки отсортированных данных примерно в три раза и около 15 % на случайных данных, при одновре-

менном уменьшении объёма индекса. Эти значения относятся к выпуску 10 и подлежат перепроверке на актуальной версии (глава 7).

Отдельного внимания заслуживает побочное следствие, отмеченное при принятии изменения: физическое содержимое страницы после воспроизведения журнальной записи стало иным, чем прежде, хотя логически страница эквивалентна. Это пример общего свойства оптимизаций страничной раскладки — они затрагивают совместимость на уровне побайтового сравнения страниц, что важно для средств проверки корректности воспроизведения журнала.

3.4 Внутривстраничное индексирование

3.4.1 Постановка

Из (2.1) и (2.4) следует, что рост ветвистости снижает число просматриваемых *страниц*, но стоимость обработки одной страницы растёт линейно по f : чтобы отобрать поддеревья для спуска, ядро вызывает `consistent` для каждого из f элементов узла. Обозначив стоимость одного вызова через c_c , получим для точечного запроса

$$T = \sum_{l=0}^{H-1} \omega_l (c_r + f c_c). \quad (3.1)$$

При достаточно большом f и нетривиальной функции `consistent` второе слагаемое доминирует. Это ограничивает осмысленное увеличение размера страницы: выигрыш от сокращения числа обращений к носителю съедается ростом процессорной стоимости просмотра узла.

3.4.2 Измерение процессорной составляющей

Соотношение слагаемых в (3.1) измерено профилированием серверного процесса на нагрузке из пространственных соединений, когда индекс и таблица целиком помещаются в буферный кэш и обращения к носителю отсутствуют (таблица 3.1).

Работа с элементами страницы составляет 46,8 %, работа с буферами —

Таблица 3.1. Распределение процессорного времени при спуске по дереву

Доля	Функция	Содержание
22,4 %	<code>gistScanPage</code>	перебор элементов страницы
14,1 %	<code>gist_point_consistent</code>	вычисление предиката
12,8 %	<code>hash_search_with_hash_value</code>	поиск страницы в таблице буферов
10,2 %	<code>heap_page_prune_opt</code>	сторона таблицы
5,7 %	<code>PinBuffer</code>	закрепление страницы
4,7 %	<code>FunctionCall5Coll</code>	вызов предиката через <code>fmggr</code>
3,3 %	<code>gistcheckpage</code>	проверка страницы
2,3 %	<code>gistdentryinit</code>	подготовка элемента

18,4 %, сторона таблицы — 15,6 %.

Из измерения следуют три вывода, определяющие приоритеты.

Механика перебора дорожке самого предиката. На цикл по элементам уходит в 1,6 раза больше времени, чем на вычисление `consistent`. Величина c_c в (3.1) в основном складывается не из предметных вычислений, а из накладных расходов обхода: разбора смещений, подготовки элемента, постановки в очередь.

Сокращение числа просматриваемых элементов выгоднее их удешевления. Устранение накладных расходов вызова через `fmggr` — например, пакетным вычислением предиката сразу для массива ключей — даёт около 5 % общего времени. Пропуск половины элементов страницы даёт около 20 %, то есть вчетверо больше. Это количественно обосновывает приоритет внутри-страничного индексирования перед оптимизацией интерфейса вызова.

Снижение перекрытия окупается и по процессору. Поиск страницы в таблице буферов и её закрепление составляют вместе 18,4 % и прямо пропорциональны числу посещённых страниц, то есть ω_∞ . Результаты главы 4 обосновывались сокращением ввода-вывода; измерение показывает, что на данных, помещающихся в оперативной памяти, они дают сопоставимый по величине процессорный выигрыш.

3.4.3 Предлагаемый подход

Разорвать эту связь можно, разместив внутри страницы вспомогательную структуру, позволяющую отбирать кандидатов для `consistent` не за

$O(f)$, а за $O(\log f)$ или за $O(m)$, где m — число реально пересекающихся с запросом элементов. Предложенная реализация [20] строит такую структуру из самих индексных кортежей: часть кортежей на странице получает дополнительную ссылку, позволяющую при несовпадении пропустить целый участок страницы, не вызывая `consistent` для его элементов («пропускающие кортежи»). Тогда (3.1) принимает вид

$$T' = \sum_{l=0}^{H-1} \omega_l (c_r + (m_l + \log f) c_c), \quad (3.2)$$

и увеличение размера страницы становится безусловно выгодным.

Требования к внутривстраничной структуре, вытекающие из ограничений обобщённого дерева поиска:

1. структура строится над теми же операциями класса операторов (`consistent`, `union`, `penalty`), то есть остаётся обобщённой;
2. её перестроение при вставке элемента должно быть дешевле, чем экономия на просмотре, иначе выигрыш на поиске оплачивается проигрышем на вставке;
3. структура должна помещаться на той же странице и корректно восстанавливаться при воспроизведении журнала;
4. при повреждении структуры результат запроса не должен становиться неверным — допустима деградация к линейному просмотру.

3.4.4 Результаты и состояние работы

Реализация, описанная в [20], повышает производительность вставки и обновления примерно в полтора раза. Ограничения, установленные там же: приём затрагивает физическую структуру индексного кортежа, а следовательно — и восстановление, поскольку появляются новые допустимые состояния страницы после сбоя, которые обязано корректно обрабатывать воспроизведение журнала; кроме того, пропускающие кортежи не используются при упорядочивающем сканировании для поиска ближайших соседей.

В основную ветку PostgreSQL результат не принят: обсуждение [21] показало, что вмешательство в формат индексного кортежа и в восстановление

требует обоснования, несоразмерного полученному выигрышу, а на нагрузках с преобладанием вставок стоимость поддержания структуры не всегда окупается (требование 2 выше).

Это единственный результат работы, опубликованный, но не доведённый до принятия в основную ветку. Согласно принципу приоритета открытой реализации (см. введение) он выносится на защиту с явной оговоркой об этом: положение подкреплено публикацией и измерениями, но не эксплуатацией.

Измерение §3.4.2 показывает, что направление выбрано верно — резерв составляет около 20 % процессорного времени, — и одновременно указывает, как снять возражение, из-за которого результат не был принят. Вспомогательная структура не обязана храниться на странице. Её можно строить при загрузке страницы в буферный кэш и держать рядом с буфером, освобождая вместе с ним и перестраивая при модификации страницы. Тогда формат индексного кортежа не меняется, журнал и восстановление не затрагиваются, совместимость с ранее построенными индексами сохраняется, а расходуется только оперативная память. Проверка этого варианта — ближайшая задача по данному направлению.

3.5 Функция штрафа и качество разбиения

Функция `penalty` определяет выбор поддерева при вставке и тем самым — перекрытие ω_l из (2.3). Стандартная для R-дерева функция минимизирует прирост площади ключа, что при многомерных данных приводит к систематическому предпочтению уже «раздутых» ключей. В [22] предложена функция штрафа, учитывающая, наряду с приростом площади, изменение периметра и степень перекрытия с соседними ключами — по мотивам критериев RR^* -дерева [4], но сформулированная в терминах интерфейса класса операторов.

Результат не был принят в основную ветку PostgreSQL: изменение функции штрафа меняет форму уже построенных индексов и потому несовместимо с требованием неизменности поведения при обновлении версии СУБД без перестроения индексов. Этот случай показателен: для промышленной СУБД ограничением оказывается не качество алгоритма, а совместимость. В главе 4

рассматривается путь, свободный от этого ограничения: воздействие на перекрытие не через функцию штрафа, а через процедуру построения индекса.

Смежный результат — корректная обработка неопределённых значений при сравнении расстояний в очереди приоритетов поиска ближайших соседей: e5d8f359610 «Fix handling Inf and Nan values in GiST pairing heap comparator».

3.6 Экспериментальная оценка

3.7 Выводы по главе

1. Отказ от обязательных функций преобразования ключа устраняет избыточные вызовы при просмотре узла и позволяет выполнять индексное сканирование без обращения к таблице, не требуя дополнительных функций класса операторов.
2. Разделение атрибутов на ключевые и покрывающие позволяет включить в индекс столбцы, тип которых не имеет класса операторов, не увеличивая высоту дерева.
3. Перезапись элемента на месте устраняет перемещение данных на странице при обновлении ключей на пути спуска.
4. Стоимость просмотра узла растёт линейно по ветвистости (3.1), что ограничивает выигрыш от роста f ; снятие этого ограничения требует внутривстраничной структуры и составляет направление дальнейшей работы.

4 ПОСТРОЕНИЕ ОБОБЩЁННОГО ДЕРЕВА ПОИСКА СОРТИРОВКОЙ

4.1 Постановка

Построение индекса последовательной вставкой (2.8) не использует того, что при построении весь набор данных известен заранее. Для В-дерева это давно учтено: индекс строится сортировкой и заполнением страниц подряд. Перенос приёма на обобщённое дерево поиска наталкивается на принципиальное препятствие: интерфейс класса операторов не содержит порядка. Он содержит операции `consistent`, `union`, `penalty`, `picksplit` — ни одна из них не задаёт линейного порядка на множестве ключей, и для большинства индексируемых типов (прямоугольники, многоугольники, сигнатуры) естественного линейного порядка не существует.

Ключевое наблюдение состоит в том, что для построения дерева *полный* порядок и не нужен. Достаточно порядка, сохраняющего локальность: если близкие в смысле индексируемого пространства объекты оказываются близкими в последовательности, то последовательное заполнение страниц даёт компактные ключи. Какой именно порядок обладает этим свойством — знает не ядро дерева, а класс операторов.

4.2 Интерфейс поддержки сортировки

Предложено расширить класс операторов необязательной функцией поддержки сортировки, задающей компаратор. Если функция определена, построение индекса выполняется по схеме:

1. все индексируемые значения преобразуются в ключи и сортируются заданным компаратором внешней сортировкой;
2. листовые страницы заполняются подряд до заданного коэффициента заполнения;
3. для каждой заполненной страницы формируется элемент вышележащего уровня с ключом `union` от её содержимого;

4. процедура повторяется по уровням до образования корня.

Существенно, что порядок объявляется в классе операторов, а не в параметрах индекса. В первой редакции работы компаратор передавался через параметр отношения, что позволяло указать произвольную функцию с подходящей сигнатурой — то есть построить заведомо некорректный индекс. Перенос в класс операторов делает выбор порядка частью определения типа индекса и допускает проверку при создании индекса.

Реализация: 16fa9b2b30a «Add support for building GiST index by sorting» (выпуск 14); проверка соответствия функции классу операторов — 6f0bc5e1daf «Fix missing validation for the new GiST sortsupport functions».

4.3 Порядок, сохраняющий локальность

Для точечного типа в качестве такого порядка использован Z-порядок (код Мортонa): координаты приводятся к целочисленному представлению, и биты координат чередуются:

$$Z(x_1, \dots, x_D) = \sum_{j=0}^{w-1} \sum_{i=1}^D b_{i,j} 2^{jD+i-1}, \quad (4.1)$$

где $b_{i,j}$ — j -й бит i -й координаты, w — разрядность координаты. Z-порядок обладает нужным свойством: точки, близкие в D -мерном пространстве, получают близкие значения Z (обратное, вообще говоря, неверно — существуют пары близких по Z точек, далёких в пространстве; для целей построения дерева это допустимо, поскольку влияет на качество, а не на корректность).

Два обстоятельства реализации заслуживают упоминания.

Представление чисел с плавающей точкой. Порядок битов представления IEEE 754, интерпретированных как целое, согласован с числовым порядком для неотрицательных значений и обращён для отрицательных. Для целей сохранения локальности этого достаточно: разрыв возникает только при переходе через ноль.

Неопределённые значения. Значения NaN не упорядочены относительно остальных, и произвольный выбор их места в порядке приводит к несогла-

сованности с семантикой `union` и `consistent`. Требуется явное соглашение, согласованное с этими функциями.

4.4 Правые ссылки при построении снизу вверх

При построении сортировкой конкурентного доступа к строящемуся дереву нет: поиск и расщепление страниц в это время невозможны. Формально это означает, что правые ссылки, введённые схемой Корнакера [5] для корректности конкурентного поиска, можно не устанавливать.

Тем не менее они устанавливаются: 265ea567852 «Set right-links during sorted GiST index build». Причина — в требовании верифицируемости: полнота цепочки правых ссылок на каждом уровне является структурным инвариантом, проверяемым средствами главы 6. Индекс, построенный сортировкой, не должен отличаться от построенного вставкой по набору выполняющихся инвариантов, иначе средство проверки будет вынуждено различать способы построения. Это общий принцип: оптимизация не должна сокращать множество проверяемых свойств структуры.

4.5 Перекрытие нелистовых ключей

4.5.1 Проблема

Наивная схема построения заполняет страницу до коэффициента заполнения и переходит к следующей, то есть выбирает точки разбиения по единственному критерию — использованию места на странице. Для одномерного случая это оптимально. Для многомерного — нет: Z-порядок сохраняет локальность лишь приближённо, и разбиение по границе страницы может рассечь плотный кластер точек так, что ключи двух соседних страниц окажутся сильно перекрывающимися.

Согласно (2.5), качество разбиения входит в стоимость поиска постоянным множителем на каждом шаге спуска. Измерение (§7.3) даёт для разбиения по заполнению страницы установившееся перекрытие $\omega_\infty \approx 4,0$ при том, что построение вставкой — та планка, которую сортированное построение должно воспроизвести, — даёт $\omega_\infty \approx 1,05$. Точечный запрос при наивной

сборке спускается примерно в четыре поддерева на каждом уровне вместо одного. Индекс строится в 6,8–8,0 раза быстрее и занимает на треть меньше места, но отвечает на запросы в 2,4–2,5 раза дороже. Именно это делает наивную сортированную сборку неприемлемой для промышленного применения и объясняет, почему приём, известный для R-деревьев, долго не переносился в обобщённое дерево поиска.

4.5.2 Алгоритм

Предложено выбирать точки разбиения не по заполнению страницы, а той же функцией `picksplit`, которая используется при расщеплении переполненного узла, то есть тем самым критерием качества, который заложен в класс операторов:

1. накапливать элементы уровня в буфере объёмом в четыре страницы;
2. применять к буферу `picksplit` рекурсивно, пока каждая часть не помещается на страницу;
3. записывать полученные части как страницы уровня.

Буфер в четыре страницы — компромисс: он достаточно велик, чтобы алгоритм разбиения мог выбрать точку разреза, не совпадающую с границей страницы, и достаточно мал, чтобы стоимость разбиения оставалась приемлемой. Ограничение метода прямо следует из этого выбора: алгоритмы разбиения с суперлинейной сложностью деградируют при четырёхкратном увеличении объёма обрабатываемых данных. Для классов операторов с такими алгоритмами поддержка сортированного построения не включается — сначала требуется улучшить их `picksplit`.

Реализация: `f1ea98a7975 «Reduce non-leaf keys overlap in GiST indexes produced by a sorted build»` (выпуск 15).

Существенно, что алгоритм устраняет не весь штраф, а его бóльшую часть: 87–89 % по измерениям §7.3. Остаточные 11–13 % — цена того, что точки разбиения выбираются в пределах буфера, а не глобально по всему уровню. Полное восстановление качества потребовало бы просмотра уровня целиком, что вернуло бы стоимость построения к порядку стоимости построения вставкой и лишило бы метод смысла. Компромисс формулируется так:

за 11–13 % качества дерева метод возвращает ускорение построения в 3,0–3,5 раза относительно построения вставкой.

4.6 Специализация компараторов

Стоимость (2.9) определяется числом сравнений, умноженным на стоимость одного сравнения. Последняя для обобщённой реализации включает косвенный вызов функции сравнения. Специализация компаратора для конкретного типа — подстановка тела функции сравнения в код сортировки — устраняет эти накладные расходы. Результаты: 53d3daa491b «Specialize intarray sorting» и 30229be755e «Simplify SortSupport for the macaddr data type».

4.7 Границы применимости

Метод применим, если для класса операторов существует порядок, сохраняющий локальность, и алгоритм разбиения не деградирует при увеличении объёма входа. Практическую проверку этих условий даёт история распространения метода на классы операторов расширения `btree_gist`, реализующего индексирование скалярных типов средствами обобщённого дерева поиска.

Первая попытка (2021) была отозвана в тот же день из-за отказов на сборочной ферме: обнаружили как ошибку в тестовом окружении, так и содержательная ошибка — компаратор для битовой строки использовал сравнение, не согласованное с семантикой типа. Возврат состоялся четырьмя годами позже, реализацией, написанной заново: e4309f73f69 «Add support for sorted gist index builds to `btree_gist`» (выпуск 18), где сортированное построение стало способом по умолчанию.

Этот эпизод — содержательный материал, а не курьёз. Он показывает, что корректность сортированного построения целиком определяется согласованностью компаратора с остальными функциями класса операторов, и что проверить эту согласованность внешними средствами затруднительно: несогласованный компаратор даёт корректно выглядящий индекс, теряющий часть ответов. Отсюда прямая связь с главой 6: инвариант «ключ родителя покрывает ключи потомков» нарушается при несогласованном компараторе

и обнаруживается проверкой структуры.

4.8 Экспериментальная оценка

Первоначальное измерение 2019 года, послужившее основанием для работы: построение индекса над тремя миллионами случайных точек заняло 32,1 с при построении вставкой и 6,1 с при построении сортировкой (выпуск 13, конфигурация по умолчанию); при этом объём индекса существенно сократился, а время выполнения запросов осталось прежним.

Это измерение относится к первой редакции и приводится как исходная мотивация. Итоговые измерения — в главе 7, с обязательным контролем перекрытия нелистовых ключей и времени выполнения запросов, а не только времени построения.

4.9 Выводы по главе

1. Задача построения обобщённого дерева поиска сводится к сортировке, если порядок, сохраняющий локальность, объявляется классом операторов через интерфейс поддержки сортировки; это делает приём применимым к произвольному классу операторов.
2. Наивное сортированное построение увеличивает перекрытие нелистовых ключей и замедляет поиск; выбор точек разбиения функцией `picksplit` над буфером в четыре страницы устраняет этот эффект.
3. Правые ссылки устанавливаются при построении, хотя конкурентности нет, — ради сохранения полного набора проверяемых инвариантов.
4. Корректность метода определяется согласованностью компаратора с остальными функциями класса операторов и должна проверяться средствами верификации структуры.

5 КОНКУРЕНТНОЕ СОПРОВОЖДЕНИЕ ИНДЕКСА

Индекс, из которого удалены записи, не освобождает занятое место сам по себе. Сборка мусора должна найти ссылки на удалённые записи, удалить их, обнаружить опустевшие страницы и вернуть их в оборот — и всё это одновременно с пользовательской нагрузкой, не блокируя её. В этой главе рассматриваются четыре составляющие задачи: порядок обхода, возврат страниц в оборот, условие безопасности переиспользования и гранулярность блокировок.

5.1 Обход в физическом порядке

Исходный алгоритм сборки мусора обходил дерево поиском в глубину. Логический порядок обхода не связан с физическим расположением страниц, поэтому обход порождает произвольное чтение (2.10).

Предложено обходить страницы индекса в порядке возрастания их номеров, как это делается для В-дерева. Тогда чтение становится последовательным, а при наличии механизма упреждающего чтения запросы к носителю выдаются пакетами.

Затруднение состоит в том, что при физическом обходе теряется информация о структуре: обрабатывая страницу, алгоритм не знает, какой странице она подчинена. Кроме того, конкурентная вставка может переместить ещё не обработанный элемент с непройденной страницы на уже пройденную (при расщеплении узла элементы уходят на новую страницу, номер которой может оказаться меньше текущей позиции обхода). Такие страницы приходится перечитывать — это слагаемое β_{cr} в (2.11).

Реализация: fe280694d0d «Scan GiST indexes in physical order during VACUUM» (выпуск 12).

5.2 Удаление и переиспользование страниц

До описываемой работы обобщённое дерево поиска не переиспользовало страницы вовсе: полностью опустевшая страница оставалась в индексе

навсегда. При нагрузке, в которой новые ключи вставляются не в ту область пространства, из которой удаляются старые, — а это типичный случай для данных с временной компонентой, — индекс рос неограниченно.

Предложен двухпроходный алгоритм:

1. первый проход — обход в физическом порядке (§5.1), в ходе которого удаляются ссылки на удалённые записи и запоминаются номера опустевших листовых страниц и номера внутренних страниц;
2. второй проход — обход запомненных внутренних страниц с поиском ссылок на опустевшие страницы; найденные ссылки удаляются, а страницы помечаются как удалённые и передаются в карту свободного пространства.

Порядок операций внутри второго прохода существенен для корректности: сначала удаляется ссылка из родителя, затем страница помечается удалённой, причём блокировки родителя и потомка сцепляются. Тогда конкурентный спуск не может увидеть ссылку на страницу, которая уже помечена удалённой. Обратный порядок допускал бы вставку в удаляемую страницу.

Внутренние страницы не удаляются, и последний потомок внутренней страницы не удаляется никогда — как и в В-дереве. Это означает, что в описанном выше неблагоприятном сценарии индекс всё ещё растёт, но на порядок медленнее.

Реализация: 7df159a620b «Delete empty pages during GiST VACUUM» (выпуск 12), уточнение проверок — 9eb5607e699 «Refactor checks for deleted GiST pages».

5.3 Условие безопасности переиспользования

Пометить страницу удалённой недостаточно, чтобы немедленно отдать её под новые данные. Сканирование, начатое до удаления, может удерживать ссылку на эту страницу и обратиться к ней позже; если страница к тому моменту переиспользована, сканирование получит чужие данные. Поэтому переиспользование разрешается лишь после того, как завершились все транзакции, которые могли начать такое сканирование.

Условие формулируется как «страница удалена раньше, чем начались все ныне активные транзакции». При хранении момента удаления в виде 32-разрядного идентификатора транзакции условие теряет смысл при зацикливании счётчика: достаточно старая страница вновь начинает выглядеть «недавно удалённой» и перестаёт быть пригодной к переиспользованию. Дефект тем неприятнее, что проявляется не сразу и не как отказ, а как постепенное прекращение возврата страниц в оборот.

Решение — хранить полный (64-разрядный) идентификатор транзакции, для которого зацикливание невозможно: 6655a7299d8 «Use full 64-bit XID for checking if a deleted GiST page is old enough». Тот же приём применён к служебным журналам транзакций: 4ed8f0913bf «Index SLRUs by 64-bit integers rather than by 32-bit integers».

Общая формулировка результата: условия безопасности, выражаемые через монотонный счётчик, должны формулироваться в терминах счётчика, не подверженного зацикливанию; иначе корректность зависит от длительности эксплуатации системы.

5.4 Гранулярность блокировок при сборке мусора

Обратный индекс (generalized inverted index) хранит для каждого ключа список ссылок на записи; при большом числе ссылок список превращается в дерево вхождений. Сборка мусора в таком дереве в исходной реализации захватывала исключительную блокировку корня дерева вхождений на всё время обработки. Для часто встречающегося ключа дерево вхождений велико, а блокировка корня останавливает все обращения к этому ключу.

Предложено два изменения:

1. исключительная блокировка захватывается только тогда, когда действительно обнаружена страница, подлежащая удалению, а не заранее;
2. блокируется не всё дерево вхождений, а поддереву, содержащее удаляемую страницу.

Реализация: 218f51584d5 «Reduce page locking in GIN vacuum» (выпуск 10); результат опубликован в [23].

5.4.1 Отмена второго изменения и её значение

Второе изменение было отменено полтора года спустя: fd83c83d094 «Fix deadlock in GIN vacuum introduced by 218f51584d5», с переносом во все поддерживаемые выпуски. Причина установлена по сообщению из эксплуатации: вставка, спускающаяся по дереву вхождений, не всегда удерживает закрепления всех страниц пути от корня к целевому листу — конкурентное расщепление родителя может это нарушить. Тогда сборка мусора, захватывающая блокировки в пределах поддерева, и вставка, продвигающаяся по дереву, образуют цикл ожидания. Неинвазивного способа устранить цикл, сохранив блокировку поддерева, найдено не было, и поведение возвращено к блокированию всего дерева вхождений.

Первое изменение сохранено: дерево блокируется только тогда, когда есть что удалять. Именно оно и даёт основной практический эффект, поскольку в большинстве проходов сборки мусора удалять нечего.

Одновременно устранён ещё один дефект того же происхождения: начиная с версии 9.4 сканирование может пропускать части дерева вхождений, спускаясь не от корня, поэтому приём «страница безопасна, так как переход вправо захватывает две страницы сразу» перестал быть достаточным, и удалённая страница могла быть переиспользована до того, как к ней обратится сканирование. Решение — дождаться завершения всех транзакций, которые могли начать такое сканирование; момент удаления записывается в неиспользуемое в индексных страницах поле заголовка, поскольку изменить формат служебной области страницы нельзя из соображений двоичной совместимости (52ac6cd2d0c «Prevent GIN deleted pages from being reclaimed too early», перенесено вплоть до 9.4). Позднее устранены ещё две взаимные блокировки того же узла кода (с6ade7a8cd3, e14641197a5).

Этот эпизод существен методологически, и в работе он приводится не как курьёз, а как результат. Алгоритм изменения гранулярности блокировок был описан, отрецензирован и опубликован [23]; рецензирование статьи подтвердило корректность рассуждения, но не могло подтвердить корректность алгоритма, поскольку нарушаемое предположение — о том, какие закрепле-

ния удерживает вставка при конкурентном расщеплении родителя, — относится не к алгоритму, а к остальной части системы. Опровергающее наблюдение пришло из эксплуатации.

Отсюда два следствия, принятые в работе как методологические принципы (см. введение): утверждение о конкурентном алгоритме считается проверенным не после рецензирования публикации, а после эксплуатации реализации; и свойства, не проверяемые функциональным тестом, должны проверяться инвариантами структуры (глава 6).

Задача удаления страниц дерева вхождений без блокирования дерева целиком оставалась открытой почти десять лет. В июле 2026 г. автором предложен полный протокол удаления [24]: опустевшие листовые страницы удаляются на ходу при обычном проходе по уровню листьев слева направо, блокировки сцепляются в том же порядке слева направо, что и всюду в обратном индексе, родитель отыскивается проходом вправо по вышележащему уровню, а страница удаляется только если её никто не удерживает закреплённой. Крайние слева и справа листья, листья, на которые указывает последняя нисходящая ссылка родителя, и внутренние страницы не удаляются — ограничение то же, что и в §5.2. Вставка, пришедшая по устаревшей нисходящей ссылке на удалённую страницу, восстанавливается переходом вправо; преждевременное переиспользование предотвращается уже существующей отметкой идентификатора транзакции (§5.3).

Существенно здесь не само предложение, а то, чем оно проверяется. К 2026 году доступны средства, которых в 2018 году не было: проверка инвариантов обратного индекса (глава 6) и точки внедрения, позволяющие останавливать выполнение в заданной точке кода. Это даёт изоляционные тесты, в которых сборка мусора приостанавливается в середине прохода, а поиск и вставка в то же дерево вхождений продолжаются, после чего структура проверяется; и тесты, провоцирующие взаимную блокировку — каждая из сторон предполагаемого цикла ожидания приостанавливается в точке своего максимального набора удерживаемых блокировок, а вторая направляется в них. Возврат к дефекту протокола блокирования должен приводить в таких тестах к зависанию.

Иными словами, к открытой задаче удалось вернуться не потому, что нашлось лучшее рассуждение, а потому, что появились средства проверки,

отсутствие которых и стало причиной неудачи 2018 года (§5.4.1). Это прямое подтверждение состоятельности принципа, принятого в работе: свойства, не проверяемые функциональным тестом, должны проверяться инвариантами структуры.

В соответствии с тем же принципом предложенный протокол в число результатов, выносимых на защиту, *не включается*: он не прошёл ни рецензирования в сообществе, ни — тем более — эксплуатации. Считать его проверенным на основании того, что рассуждение выглядит убедительно, означало бы повторить ошибку 2018 года внутри работы, посвящённой разбору этой ошибки. Направление остаётся в перспективах.

5.5 Предикатные блокировки

Сериализуемая изоляция моментальных снимков [8] требует обнаружения конфликта между транзакцией, прочитавшей некоторый диапазон, и транзакцией, вставившей в этот диапазон новое значение. Для этого читающая транзакция ставит предикатную блокировку на прочитанную область, а пишущая — проверяет наличие таких блокировок. Без поддержки со стороны метода доступа блокировка ставится на отношение целиком, что порождает ложные конфликты.

Для обобщённого дерева поиска гранулярность естественно выбрать страничной: сканирование блокирует каждую просмотренную страницу, вставка проверяет блокировку страницы, на которую помещается элемент. Такая гранулярность корректна, поскольку ключ страницы покрывает все содержащиеся в ней элементы: если новое значение помещается на страницу, оно попадает в область, покрытую ключом, то есть в область, которую могло прочитать сканирование.

Тонкость реализации — выбор точек постановки и проверки: проверка не должна выполняться при построении индекса и должна корректно обрабатывать вставку ссылки на новую страницу при расщеплении.

Реализация: `Zad55863e93` «Add predicate locking for GiST» (выпуск 11).

Для обратного индекса потребовался пересмотр принципа. Выявлены две ошибки: во-первых, при включении отложенной вставки на ходу последующие вставки в список отложенных не конфликтовали с ранее поставлен-

ными блокировками страниц словаря; во-вторых, сканирование, не нашедшее совпадений, не ставило блокировку на страницу словаря, хотя смысл предикатной блокировки — заблокировать *промежуток* между значениями независимо от наличия совпадений. Решение: сканирование всегда блокирует страницу метаданных (эта блокировка представляет просмотр списка отложенных вставок вне зависимости от того, существует ли он в данный момент) и всегда блокирует страницу словаря, даже при отсутствии совпадений. Одновременно снято излишнее блокирование: поскольку все элементы дерева вхождений относятся к одному значению ключа, достаточно блокировки корня этого дерева.

Реализация: 0bef1c0678d «Re-think predicate locking on GIN indexes» (выпуск 11); принцип зафиксирован в описании структуры, а не только в коде, — чтобы последующие изменения проверялись на соответствие ему.

5.6 Согласованность индекса при конкурентном создании

Создание индекса без блокирования пишущих транзакций выполняется в три фазы с ожиданием завершения транзакций между ними. Выявлено, что данные транзакций, подготовленных к двухфазной фиксации, при этом могли не попасть в индекс: процедура ожидания не учитывала их состояния. Дефект приводил к молчаливой неполноте индекса. Исправления: 8a54e12a38d, 3cd9c3b9219, fdd965d074d (2021, с переносом в поддерживаемые выпуски). Материал доклада на PGCon 2022.

5.7 Упреждающее чтение

Из (2.11) следует, что выигрыш от физического порядка обхода пропорционален разности $c_r - c_s$. На современных накопителях эта разность сокращается, но появляется другой источник выигрыша: зная последовательность страниц заранее, можно выдавать несколько запросов чтения одновременно, скрывая задержку носителя за счёт параллелизма.

Механизм потокового чтения применён к сборке мусора в трёх методах

доступа: c5c239e26e3 «Use streaming read I/O in btree vacuuming», 69273b818b1 «Use streaming read I/O in GiST vacuuming», e215166c9c8 «Use streaming read I/O in SP-GiST vacuuming» (выпуск 18). Страницы, требующие повторной обработки из-за конкурентных вставок (β в (2.11)), читаются обычным образом, поскольку их последовательность заранее неизвестна.

5.8 Экспериментальная оценка

Сравниваются сборки, различающиеся ровно одним коммитом; первичной величиной взяты счётчики, а не времена (обоснование — ниже).

5.8.1 Порядок обхода

Пара версий вокруг fe280694d0d. Пять миллионов точек, удаляется каждая пятая строка, индекс около 45 400 страниц, `shared_buffers` = 128 МБ. Перед каждым замером сбрасывается страничный кэш операционной системы, после чего таблица прогревается просмотром: холодными остаются только страницы индекса, иначе просмотр таблицы, одинаковый у обеих версий, размывает эффект.

Число читаемых страниц индекса от условий не зависит и составляет 90 900 против 45 400 при индексе в 45 400 страниц: отношение ровно 2,00 во всех прогонах. Обход в глубину читает каждую страницу дважды, физический — по разу.

Время — величина иной природы, и смешивать их нельзя. Счётчик `idx_blks_read` считает промахи мимо `shared_buffers`, а не обращения к носителю: повторное чтение страницы при логическом обходе почти всегда попадает в страничный кэш операционной системы. Число запросов к носителю у обеих версий поэтому примерно одинаково; различается *последовательность* этих запросов, а можно ли ею воспользоваться, определяется упреждающим чтением.

Таблица 5.1 выделяет механизм явления. При отключённом упреждении выигрыша практически нет (1,09), хотя счётчики промахов по-прежнему различаются вдвое: обе версии выполняют по одному синхронному чтению на страницу, стоимость страницы от порядка не зависит, и выигрывать фи-

Таблица 5.1. Время очистки индекса в зависимости от упреждающего чтения. Медианы трёх прогонов

read_ahead_kb	логический обход, с	физический, с	выигрыш
0	57,8	52,8	1,09
128 (по умолчанию)	20,9	6,1	3,43
4096	16,7	6,2	2,69

зическому порядку нечем. При включённом упреждении последовательное обращение превращает 45 400 чтений по 8 КБ примерно в 2 800 чтений по 128 КБ, тогда как разбросанные обращения логического порядка от упреждения выигрывают мало и часто читают впустую. Это ровно различие c_r и c_s из (2.10) и (2.11), проявленное изменением одной настройки.

Попутно: время физического обхода воспроизводится с точностью до 0,2 с (6,1 / 6,1 / 6,1), логического — нет (29,6 / 20,2 / 20,9). И очень большое упреждение помогает логическому порядку, а не физическому: при 4096 КБ первый ускоряется с 20,9 до 16,7, а второй остаётся на 6,2, упираясь в полосу пропуска (355 МБ за 6,1 с, около 58 МБ/с).

Использованный носитель — сетевой виртуальный диск, то есть наименее благоприятный для этого изменения случай. Дж. Джейнс, откликнувшийся на коммит через две недели после его принятия [25], сообщил о 50-кратном сокращении времени сборки мусора на жёстком диске, 30-кратном на SSD ноутбука и 3-кратном на сетевом томе AWS GP2. Наши 3,43 на сетевом диске согласуются с его 3 на сетевом томе. Весь разброс от 1,09 до 50 определяется одним отношением c_r/c_s : большим у диска с подвижной головкой, умеренным у локального SSD, малым у сетевого тома и равным единице при отключённом упреждении. Совпадение независимо полученного трёхкратного значения с нашим подтверждает модель лучше, чем любое из этих чисел по отдельности.

Следует указать и то, чего воспроизвести не удалось. Потокочное чтение (§5.7) на этом оборудовании измеримого эффекта не дало: шесть прогонов пары версий вокруг 69273b818b1 дали медиану 52,5 с с обеих сторон. При упреждении по умолчанию ядро операционной системы и без того читает на шестнадцать страниц вперёд при последовательном просмотре, то есть выполняет ту же работу другим способом. Механизм должен давать выигрыш

там, где на упреждение полагаться нельзя, — в частности для страниц β , чью последовательность упреждение угадать не может, — но случая, в котором это измерено, у нас нет.

5.8.2 Возврат страниц в оборот

Пара версий вокруг 7df159a620b; родитель уже содержит физический обход, так что пара изолирует именно возврат страниц. Нагрузка — та, ради которой изменение делалось: окно шириной в три раунда движется по оси, за раунд вставляется 200 000 строк в новую область и удаляются все строки, вышедшие из окна, после чего выполняется сборка мусора. С третьего раунда число живых строк постоянно и равно 600 000.

До изменения объём индекса растёт линейно при постоянном числе живых строк — около 1 640 страниц за раунд, безостановочно. После изменения рост выходит на плато: с шестого раунда прирост падает примерно до 44 страниц за раунд, то есть в 37 раз.

Плато при этом лежит не на идеальной отметке: 600 000 живых строк занимают около 4 900 страниц, а установившийся объём — около 8 400. Разница есть цена неполноты возврата (§5.2): внутренние страницы и последний потомок не удаляются, поэтому остаточный прирост не нулевой. Результат формулируется соответственно: неограниченный рост заменён медленным, а не устранён.

5.8.3 Время ожидания блокировок в обратном индексе

5.9 Выводы по главе

1. Обход индекса в физическом порядке страниц при сборке мусора заменяет произвольное чтение последовательным; конкурентные вставки требуют повторной обработки части страниц.
2. Двухпроходный алгоритм с удалением ссылок при сцеплённых блокировках возвращает опустевшие страницы в оборот и ограничивает неограниченный рост индекса.

3. Условие безопасности переиспользования страницы, выраженное через полный идентификатор транзакции, не теряет смысла при длительной эксплуатации.
4. Отложенный захват исключительной блокировки сокращает время ожидания при сборке мусора в обратном индексе; сужение области блокирования до поддерева при действующем протоколе закреплений некорректно (§5.4.1) и требует его пересмотра.
5. Страничная гранулярность предикатных блокировок достаточна для корректности сериализуемой изоляции над обобщённым деревом поиска; для обратного индекса требуется дополнительно блокировать страницу метаданных и страницу словаря при отсутствии совпадений.

6 ВЕРИФИКАЦИЯ СТРУКТУРНЫХ ИНВАРИАНТОВ ИНДЕКСА

6.1 Постановка

Ошибка в реализации метода доступа проявляется иначе, чем ошибка в прикладной программе. Индекс не отказывает — он начинает отвечать неполно: запрос, которому соответствуют существующие записи, их не находит. Приложение при этом работает, и расхождение обнаруживается спустя произвольное время, когда восстановить причину уже невозможно.

Главы 4 и 5 дают три примера, в которых дефект относился именно к этому классу: несогласованный компаратор при сортированном построении (§4.7), преждевременный возврат удалённых страниц в оборот (§5.4), неучёт подготовленных транзакций при конкурентном создании индекса. Ни один из них не выявляется функциональным тестированием: результат каждой отдельной операции корректен, нарушается свойство структуры в целом.

Отсюда постановка: сформулировать структурные инварианты индекса и построить процедуру их проверки, применимую не только при отладке, но и в промышленной эксплуатации — то есть на действующей системе, без монопольного доступа к индексу, с ограниченным расходом памяти и с возможностью выполнения на реплике. Подход к отладке индексных структур, от которого отталкивается настоящая глава, изложен в [17].

6.2 Инварианты обобщённого дерева поиска

Для обобщённого дерева поиска проверяемые свойства формулируются в терминах интерфейса класса операторов, что делает проверку применимой к любому классу:

1. *Согласованность ключа родителя.* Для каждого элемента внутренней страницы ключ этого элемента покрывает ключи всех элементов страницы, на которую он ссылается: $\text{consistent}(key_{parent}, key_{child})$ истинно для всех потомков. Нарушение означает, что поиск не спустится в

поддерево, содержащее ответ, — то самое молчаливое неполное выполнение запроса.

2. *Сбалансированность*. Все листовые страницы находятся на одной глубине; внутренняя страница ссылается либо только на внутренние, либо только на листовые страницы.
3. *Полнота цепочки правых ссылок*. Правая ссылка страницы указывает на страницу того же уровня; проход по цепочке правых ссылок посещает все страницы уровня ровно один раз.
4. *Согласованность соседних ссылок*. Правая ссылка левого соседа указывает на страницу, левая ссылка которой указывает обратно на левого соседа.
5. *Недостижимость удалённых страниц*. На страницу, помеченную удалённой, не ссылается ни один элемент внутренней страницы.

Инвариант 3 объясняет решение §«Правые ссылки при построении снизу вверх»: если бы построение сортировкой оставляло правые ссылки неустановленными, проверка была бы вынуждена различать способы построения индекса.

6.3 Проверка согласованности ключей родителя

Прямая проверка инварианта 1 требует для каждой страницы знать ключ ссылающегося на неё элемента. Обход сверху вниз даёт это знание естественно, но порождает произвольное чтение и удержание состояния, пропорционального ширине уровня. Применяемая схема: обход в порядке уровней с передачей ключей родителей вниз через промежуточное хранилище, что ограничивает расход памяти шириной одного уровня.

Для обратного индекса структура сложнее: это не дерево, а граф — дерево словаря, в листьях которого располагаются либо списки вхождений, либо корни деревьев вхождений. Проверяемые свойства формулируются соответственно: согласованность ключей вдоль путей графа, наличие хотя бы одной нисходящей ссылки у внутренней страницы, однородность уровня, на

который ссылается внутренняя страница, а также упорядоченность элементов внутри страницы.

Реализация: d70b17636dd «amcheck: Move common routines into a separate module», 14ffaec0fb «amcheck: Add gin_index_check() to verify GIN index» (выпуск 18). Последующие исправления — 0cf205e122a (проверки дерева вхождений), cdd1a431f21 (проверка ключа родителя), 0b54b392334 (порядок элементов словаря) — показывают, что и сама процедура проверки нуждается в верификации: ложноположительное срабатывание средства контроля целостности не менее вредно, чем пропуск дефекта, поскольку подрывает доверие к средству.

Верификация обобщённого дерева поиска реализована частично и остаётся в работе.

6.4 Обход правых ссылок со сцеплением блокировок

Инвариант 4 нельзя проверить, удерживая блокировку одной страницы: требуется одновременно видеть левого соседа и правого. Это означает сцепление блокировок — приём, которого средства проверки целостности до того избегали, поскольку он усложняет рассуждение о возможности взаимных блокировок.

Обоснование его допустимости: блокировки захватываются строго в одном направлении (слева направо), что исключает цикл ожидания; проверка выполняется в разделяемом режиме и не препятствует чтению. Практический аргумент — проверка соседних ссылок обнаруживает значительную долю реально встречающихся повреждений при малых накладных расходах, поэтому усложнение оправдано.

Реализация: 39132b784ae «Teach amcheck to verify sibling links in all cases» (выпуск 13). Отмечено, что на реплике корректность проверки зависит от согласованности воспроизведения журнала: без соответствующих исправлений возможны ложные срабатывания.

6.5 Нормализация индексных кортежей

Сравнение индексного кортежа с кортежем, построенным заново из значения таблицы, — основной приём проверки соответствия индекса данным. Побайтовое сравнение здесь неприменимо: одно и то же значение переменной длины может быть представлено по-разному (со сжатием и без, с заголовком разной длины), причём представление зависит от истории операций, а не от значения.

Введена нормализация кортежа перед сравнением, приводящая все представления к каноническому виду: b1fe8efdf17 «amcheck: Normalize index tuples containing uncompressed varlena», ab65dfb0fb2 «amcheck: Support for different header sizes of short varlena datum». Позднее выявлено использование некорректного средства определения размера: da1eff08a5b «amcheck: Use correct varlena size accessor in bt_normalize_tuple()».

6.6 Проверка на реплике и в процессе восстановления

Требование выполнимости проверки на реплике имеет практическое основание: проверка нагружает подсистему ввода-вывода, и выполнять её на системе, обслуживающей пользователей, нежелательно.

Ограничения реплики: журналируемые не полностью отношения на реплике не имеют осмысленного содержимого, и проверять их бессмысленно (6754fe65a4c «amcheck: Skip unlogged relations during recovery»); кроме того, проверка, требующая согласованного среза данных, должна корректно получать его в условиях восстановления (1f28982e409 «amcheck: Fix snapshot usage in bt_index_parent_check»).

6.7 Коды ошибок повреждения данных

Обнаруженное повреждение должно быть отличимо программно, а не по тексту сообщения: средства эксплуатации должны уметь автоматически распознать повреждение и, например, вывести узел из обслуживания. Для

этого сообщением о повреждении присвоены отдельные коды ошибок: 8ec97e78a77 «Add error codes when vacuum discovers VM corruption». Это часть более общей задачи, поставленной в [17]: сделать повреждение наблюдаемым событием, а не аномалией, обнаруживаемой пользователем.

6.8 Экспериментальная оценка

6.9 Выводы по главе

1. Структурные инварианты обобщённого дерева поиска и обратного индекса формулируются в терминах интерфейса класса операторов и потому проверяемы для произвольного класса операторов.
2. Проверка согласованности ключей родителя с обходом по уровням ограничивает расход памяти шириной уровня и допускает выполнение на действующей системе.
3. Проверка соседних ссылок требует сцепления блокировок; однонаправленный порядок их захвата исключает взаимные блокировки.
4. Сравнение индексного кортежа с эталонным требует предварительной нормализации представления значений переменной длины.
5. Выполнение проверки на реплике снимает нагрузку с обслуживающей системы и требует корректной работы в условиях восстановления.

7.1 Методика измерений

Требования к измерениям, принятые в работе:

1. Все результаты переснимаются на едином актуальном выпуске СУБД. Значения, приведённые в главах 3–6 со ссылкой на публикации 2017–2018 гг., относятся к выпускам того времени и служат исторической мотивацией, а не итоговым результатом.
2. Сравниваются сборки, различающиеся *только* исследуемым изменением; базовая точка — ближайший предшествующий коммит.
3. Для каждой конфигурации выполняется не менее пяти прогонов; приводится медиана и разброс.
4. Одновременно с целевой метрикой измеряются метрики, которые изменение могло ухудшить. Для построения индекса это обязательно означает измерение времени выполнения запросов и перекрытия ключей — согласно §4.5, ускорение построения способно замедлить поиск.
5. Указываются: версия, параметры сборки, параметры конфигурации, отличные от умолчаний, характеристики оборудования и файловой системы, состояние кэшей перед прогоном.

7.2 Наборы данных

Набор	Тип	Назначение
Равномерные точки	point	базовый случай, $D = 2$
Кластеризованные точки	point	чувствительность к качеству разбиения
Географические объекты	box, polygon	прикладной случай
Диапазоны дат	tsrange	односторонняя вставка, сборка мусора
Многомерные кубы	cube	зависимость от D
Целочисленные массивы	int[]	обратный индекс, сигнатуры
Скалярные типы	btree_gist	границы применимости сортировки

Размерность варьируется от 2 до 32 для проверки следствий (2.7); объём — от помещающегося в кэш буферов до превышающего его на порядок, чтобы разделить процессорную и введено-выводную составляющие стоимости.

7.3 Перекрытие ключей при разных стратегиях разбиения

Сопоставлялись две сборки PostgreSQL, различающиеся ровно одним изменением (f1ea98a7975): разбиение по заполнению страницы и разбиение функцией `picksplit` на буфере в четыре страницы. Метрика — число обращений к страницам индекса на точечный запрос; окно запроса масштабировалось как $0,5/\sqrt{N}$, чтобы ожидаемое число совпадений не зависело от объёма. Индекс целиком помещался в буферный кэш, поэтому измерялась работа алгоритма, а не носителя.

Построение вставкой измерено на той же сборке: из семейства операторов удалена опорная функция поддержки сортировки, после чего ядро выбирает обычный путь вставки. Тем самым сравниваются алгоритмы, а не версии.

Построение вставкой даёт дерево, близкое к оптимальному: ω_l составляет 1,0–1,13 на всех уровнях, то есть точечный запрос спускается ровно по одному пути. Относительно этой планки разбиение по заполнению страницы удорожает запрос в 2,5 и 2,4 раза, а разбиение функцией класса операторов — на 17 и 18 %, то есть устраняет 89 и 87 % штрафа. Ускорение построения от-

Таблица 7.1. Три способа построения, равномерное распределение

N	способ построения	сборка, с	страниц	обращений/запрос
10^6	вставка	4,38	9 115	3,95
10^6	сортировка, по заполнению	0,64	6 063	10,01
10^6	сортировка, picksplit	1,48	8 165	4,62
10^7	вставка	69,61	90 908	5,19
10^7	сортировка, по заполнению	8,65	60 608	12,23
10^7	сортировка, picksplit	19,80	81 277	6,10

носительно вставки составляет 6,8–8,0 раза при разбиении по заполнению и 3,0–3,5 раза при разбиении picksplit.

Отношение между двумя стратегиями сортированного построения устойчиво: 2,17 при 10^6 и 2,00 при 10^7 ; на кластеризованных данных с размером кластера от $0,5f$ до $8f$ — 2,1–2,8.

Побочный результат, противоречащий обычному ожиданию «лучше качество — больше индекс»: сортированное построение даёт индекс *меньшего* объёма, чем построение вставкой (на 33 % при разбиении по заполнению и на 11 % при picksplit), поскольку расщепление оставляет страницы заполненными примерно на 70 %, а последовательная упаковка — на заданный коэффициент заполнения.

Поуровневые значения ω_l получены восстановлением ключей снизу вверх: для листа — описывающий прямоугольник его элементов, для внутреннего узла — объединение ключей потомков (при построении сортировкой хранимый ключ равен в точности этому объединению).

Таблица 7.2. Перекрытие по уровням, $N = 10^7$, 2000 зондов

Уровень	вставка		по заполнению		picksplit	
	узлов	ω_l	узлов	ω_l	узлов	ω_l
0 (корень)	1	1,00	1	1,00	1	1,00
1	8	1,05	3	1,78	5	1,99
2	823	1,13	363	4,24	601	2,47
3 (листья)	90 076	1,06	60 241	3,95	80 737	1,35

Величина ω_l выходит на установившееся значение и далее с глубиной не растёт: $\omega_\infty \approx 4,0$ при разбиении по заполнению и $\omega_\infty \approx 1,3$ при разби-

нии функцией класса операторов. Верхние уровни исключение только потому, что там ω_l ограничено числом узлов. Это подтверждает (2.5) и объясняет устойчивость двукратного отношения при изменении объёма на порядок.

7.4 Построение индекса

7.5 Поиск

7.6 Сопровождение

7.7 Верификация

7.8 Сопоставление с аналитическими моделями

7.9 Границы применимости

Из полученных результатов и из хода работы следуют ограничения, которые формулируются здесь явно:

1. Сортированное построение требует порядка, сохраняющего локальность, и алгоритма разбиения, не деградирующего при четырёхкратном увеличении объёма входа (§4.5).
2. Выигрыш от физического порядка обхода пропорционален разности стоимостей произвольного и последовательного чтения и сокращается на накопителях с малой задержкой, частично компенсируясь упреждающим чтением (§5.7).
3. Возврат страниц в оборот не устраняет рост индекса полностью: внутренние страницы и последний потомок внутренней страницы не удаляются (§5.2).
4. При большой размерности данных перекрытие ключей растёт вне зависимости от качества алгоритмов (2.7); вне некоторого D древовидное индексирование уступает последовательному просмотру.

5. Изменение функции штрафа несовместимо с требованием работоспособности ранее построенных индексов после обновления версии СУБД (§3.5).

7.10 Внедрение

7.10.1 Основная ветка СУБД PostgreSQL

Все алгоритмы, описанные в главах 3–6, приняты в основную ветку PostgreSQL и поставляются в составе выпусков 10–19. Перечень изменений с идентификаторами приведён в приложении А.

Существенны две особенности этой формы внедрения. Во-первых, каждое изменение прошло независимое рецензирование участниками сообщества разработчиков, и переписка по каждому доступна публично, то есть процесс принятия результата документирован. Во-вторых, результат распространяется вместе с СУБД, а значит доступен всем её пользователям без дополнительных действий; это делает эксперименты воспроизводимыми на общедоступных выпусках без обращения к авторской реализации.

7.10.2 Промышленная эксплуатация

Результаты эксплуатируются в облачных сервисах управляемых баз данных, где проверены на нагрузках, недостижимых в лабораторных условиях по объёму и длительности. Именно такая эксплуатация выявила практическую значимость результатов §5.2 и §5.3: неограниченный рост индекса и прекращение возврата страниц в оборот проявляются на горизонте месяцев непрерывной работы.

7.10.3 Учебный процесс

Материалы работы используются в учебном процессе УрФУ при подготовке бакалавров и магистров по направлению «Информатика и вычислительная техника», а также в справочной литературе по внутреннему устройству СУБД [15], где описаны, в частности, покрывающие атрибуты индекса и построение обобщённого дерева поиска сортировкой.

7.10.4 Предшествующая открытая реализация

Первой программной реализацией методов доступа, развитых в настоящей работе, была библиотека индексирования многомерных классифицированных данных, зарегистрированная как открытый электронный ресурс в 2009 году [16]. Распространения она не получила: в то время отсутствовала общепринятая инфраструктура публикации исходного кода, и регистрация в отраслевом фонде алгоритмов и программ обеспечивала формальную открытость, но не доступность. Настоящая работа доводит тот же принцип до практического результата: реализация распространяется не как отдельная библиотека, а как часть СУБД, и потому доступна без каких-либо действий со стороны пользователя.

7.11 Выводы по главе

ЗАКЛЮЧЕНИЕ

В работе решена задача снижения вычислительной и введено-выводной стоимости построения, поиска и сопровождения индексов многомерных данных за счёт развития моделей и алгоритмов обобщённых деревьев поиска. Основные результаты:

1. Построены аналитические модели стоимости операций над обобщённым деревом поиска (глава 2), связывающие ветвистость узла, перекрытие ключей и размерность данных с числом обращений к страницам. Из моделей следует, что перекрытие ключей не накапливается с глубиной дерева, а выходит на характерное для стратегии разбиения установившееся значение, и потому входит в стоимость поиска постоянным множителем на каждом шаге спуска; ветвистость же влияет на стоимость поиска логарифмически, а на объём индекса и стоимость массовых операций — обратно пропорционально. Отсюда определены три параметра, поддающиеся воздействию без изменения класса операторов: ветвистость, перекрытие и порядок обращения к страницам.
2. Разработаны алгоритмы повышения ветвистости узла (глава 3): отказ от обязательных функций преобразования ключа с приданием их отсутствию семантики тождественного представления; разделение атрибутов индекса на ключевые и покрывающие, позволяющее включить в индекс столбцы, тип которых не имеет класса операторов, не увеличивая высоту дерева; перезапись элемента на странице без перемещения данных. Показано, что стоимость просмотра узла растёт линейно по ветвистости, что ограничивает выигрыш от её роста и требует внутри-страничного индексирования.
3. Разработан алгоритм построения обобщённого дерева поиска сортировкой (глава 4). Порядок, сохраняющий локальность, объявляется классом операторов через интерфейс поддержки сортировки, что делает приём применимым к произвольному классу операторов, а не только к R-дереву. Выявлен и устранён основной побочный эффект метода — рост

перекрытия нелистовых ключей: точки разбиения выбираются функцией разбиения узла над буфером в четыре страницы, а не по границе заполнения страницы. Определены границы применимости метода.

4. Разработаны алгоритмы конкурентного сопровождения индекса (глава 5): обход в физическом порядке страниц вместо логического; двухпроходное удаление опустевших страниц с возвратом их в оборот; формулировка условия безопасности переиспользования через полный идентификатор транзакции, не подверженный заикливанию; сужение области и отложенный захват блокировок при сборке мусора в обратном индексе; страничная гранулярность предикатных блокировок для сериализуемой изоляции.
5. Разработан метод верификации структурных инвариантов индексных структур (глава 6), пригодный для промышленной эксплуатации: инварианты формулируются в терминах интерфейса класса операторов; проверка согласованности ключей родителя выполняется с расходом памяти порядка ширины уровня; проверка соседних ссылок выполняется со сцеплением блокировок в однонаправленном порядке; сравнение кортежей выполняется после нормализации представления; проверка допускает выполнение на реплике.
6. Все перечисленные алгоритмы реализованы в СУБД PostgreSQL, приняты в её основную ветку и поставляются в составе выпусков 10–19 (приложение А). Проведена экспериментальная оценка эффективности и установлены границы применимости (глава 7).

Рекомендации и перспективы дальнейшей разработки темы

- *Внутристраничное индексирование (§3.4).* Снятие линейной по ветвистости составляющей стоимости просмотра узла сделало бы увеличение размера страницы безусловно выгодным и изменило бы баланс параметров, установленный главой 2. Основное затруднение — стоимость поддержания внутристраничной структуры при вставках.

- *Верификация обобщённого дерева поиска* в полном объёме (глава 6): для обратного индекса проверка реализована, для обобщённого дерева поиска работа не завершена.
- *Слияние страниц при сборке мусора* для В-дерева: возврат страниц в оборот, реализованный в главе 5 для обобщённого дерева поиска, не решает задачу частично заполненных страниц.
- *Удаление страниц дерева вхождений обратного индекса без избыточных блокировок* — продолжение §5.4.
- *Параллельное построение индекса*: сортировка, к которой сведено построение (глава 4), распараллеливается естественным образом, и это направление остаётся неисследованным для обобщённого дерева поиска.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- [1] Hellerstein Joseph M, Naughton Jeffrey F, Pfeffer Avi. Generalized search trees for database systems. — University of Wisconsin-Madison, 1995.
- [2] Guttman Antonin. R-trees: a dynamic index structure for spatial searching // ACM SIGMOD Record / ACM. — 1984. — Vol. 14. — P. 47–57.
- [3] The R*-tree: an efficient and robust access method for points and rectangles / Beckmann Norbert, Kriegel Hans-Peter, Schneider Ralf, and Seeger Bernhard // ACM SIGMOD Record / ACM. — 1990. — Vol. 19. — P. 322–331.
- [4] Beckmann Norbert, Seeger Bernhard. A revised R*-tree in comparison with related index structures // Proceedings of the 2009 ACM SIGMOD International Conference on Management of Data / ACM. — 2009. — P. 799–812.
- [5] Kornacker Marcel, Mohan C, Hellerstein Joseph M. Concurrency and recovery in generalized search trees // ACM SIGMOD Record / ACM. — 1997. — Vol. 26. — P. 62–72.
- [6] Lehman Philip L, Yao S Bing. Efficient locking for concurrent operations on B-trees // ACM Transactions on Database Systems. — 1981. — Vol. 6, no. 4. — P. 650–670.
- [7] PostgreSQL: the suitable DBMS solution for astronomy and astrophysics / Chilingarian Igor, Bartunov Oleg, Richter Janko, and Sigaev Teodor // Astronomical Data Analysis Software and Systems (ADASS) XIII. — 2004. — Vol. 314. — P. 225.
- [8] Ports Dan R K, Grittnner Kevin. Serializable snapshot isolation in PostgreSQL // Proceedings of the VLDB Endowment. — 2012. — Vol. 5. — P. 1850–1861.
- [9] Бородин А. М., Поршнева С. В., Сидоров М. А. Использование пространственных индексов для обработки аналитических запросов и агрегирования многомерных данных в ИАС // Известия Томского политехнического университета. — 2008. — Vol. 313, no. 5. — P. 64–67.

- [10] Бородин А. М., Поршнева С. В. Аналитические способы оценки эффективности применения пространственных индексов в OLAP-системах // Научно-технические ведомости СПбГПУ. Информатика. Телекоммуникации. Управление. — 2011. — no. 2 (120). — P. 93–100.
- [11] Бородин А. М., Поршнева С. В. О параллельном построении пространственных индексов основной памяти в OLAP-системах // Научно-технические ведомости СПбГПУ. Информатика. Телекоммуникации. Управление. — 2011. — no. 1 (97). — P. 72–74.
- [12] Бородин А. М. Алгоритмы быстрого доступа к многомерным данным в OLAP-системах. — LAP Lambert Academic Publishing, 2012. — P. 180.
- [13] Бородин А. М. Разработка быстрых алгоритмов доступа к многомерным данным в OLAP-системах : Дис. канд. техн. наук ; Сибирский государственный университет телекоммуникаций и информатики. — 2011.
- [14] Borodin Andrey, Mirvoda Sergey, Porshnev Sergey. Analysis of multidimensional data with high dimensionality: data access problems and possible solutions // ITM Web of Conferences. — 2016. — Vol. 10.
- [15] Рогов Е. В. PostgreSQL 18 изнутри. — М. : ДМК Пресс, 2026. — P. 682. — ISBN: 978-5-93700-499-4.
- [16] Бородин А. М., Поршнева С. В., Мирвода С. Г. Программная библиотека «Индексирование многомерных классифицированных данных». — Свидетельство о регистрации электронного ресурса ОФЭРНИО № 14197 от 2 ноября 2009 г.; инв. номер ВНИТИЦ 45392457.00084-01 99 01. — 2009.
- [17] Borodin Andrey, Mirvoda Sergey, Porshnev Sergey. Database index debug techniques: a case study // Beyond Databases, Architectures and Structures (BDAS) / Springer. — 2016. — P. 648–658.
- [18] Angelakos Jimmy. PostgreSQL Mistakes and How to Avoid Them. — Shelter Island, NY : Manning Publications, 2025. — ISBN: 978-1-63343-687-9.

- [19] Optimization of memory operations in generalized search trees of PostgreSQL / Borodin Andrey, Mirvoda Sergey, Kulikov Ilia, and Porshnev Sergey // Beyond Databases, Architectures and Structures (BDAS) / Springer. — 2017. — Communications in Computer and Information Science. — P. 224–232.
- [20] Borodin Andrey, Mirvoda Sergey, Porshnev Sergey. Intra-page Indexing in Generalized Search Trees of PostgreSQL // Data Analytics and Management in Data Intensive Domains (DAMDID/RCDL 2019), Kazan, Russia. — 2019. — Vol. 2523 of CEUR Workshop Proceedings. — Access mode: <https://ceur-ws.org/Vol-2523/paper17.pdf>.
- [21] Borodin Andrey et al. [WiP] GiST intrapage indexing. — Discussion on the postgresql-hackers mailing list. — 2018. — Access mode: <https://postgres/m/7780A07B-4D04-41E2-B228-166B41D07EEE@yandex-team.ru>.
- [22] Borodin Andrey, Mirvoda Sergey, Porshnev Sergey. Improving penalty function of R-tree over generalized index search tree // IEEE 2nd International Conference on Big Data Analysis (ICBDA) / IEEE. — 2017.
- [23] Improving generalized inverted index lock wait times / Borodin Andrey, Mirvoda Sergey, Porshnev Sergey, and Bakunov Ilia // Journal of Physics: Conference Series. — 2018. — Vol. 1015.
- [24] Borodin Andrey. Delete GIN posting tree pages without excessive locking. — postgresql-hackers mailing list. — 2026. — July. — <https://postgres/m/DDD3E964-5047-499C-8208-594F182D140D%40yandex-team.ru>.
- [25] Janes Jeff. Re: GiST VACUUM. — postgresql-hackers mailing list. — 2019. — Mar. — https://postgres/m/CAMkU%3D1w0qJgjXMf0_R_rUjZ6dvba3x-xeCPBLYQZ1pTV1utLow%40mail.gmail.com.

А ПЕРЕЧЕНЬ РЕЗУЛЬТАТОВ, ПРИНЯТЫХ В ОСНОВНУЮ ВЕТКУ POSTGRESQL

Столбец «Выпуск» указывает первый выпуск PostgreSQL, в котором результат стал доступен пользователям. Пометка «→ N» означает, что изменение было дополнительно перенесено в поддерживаемые выпуски начиная с N-го, то есть доступно и в более старых версиях. Столбец «Гл.» отсылает к главе, в которой результат изложен.

Выпуски PostgreSQL появляются ежегодно; выпуск 10 вышел в 2017 году, выпуск 18 — в 2025 году, выпуск 19 находится в разработке.

Коммит	Дата	Изменение	Выпуск	Гл.
b1328d78f88	2016-09-09	Перезапись элемента на странице без перемещения данных	10	3
218f51584d5	2017-03-23	Снижение блокирования при сборке мусора в обратном индексе	10	5
d3a4f89d8a3	2017-09-19	Необязательные функции преобразования ключа	11	3
de1d042f597	2017-11-20	Индексное сканирование без обращения к таблице для cube и seg	11	1
3ad55863e93	2018-03-27	Предикатные блокировки для обобщённого дерева поиска	11	5
0bef1c0678d	2018-05-04	Пересмотр предикатных блокировок обратного индекса	11	5
f919c165ebd	2018-08-30	Проверка предельной размерности во всех конструкторах cube	12	1
2a6368343ff	2018-09-19	Поиск ближайших соседей в дереве с разбиением пространства	12	1
fd83c83d094	2018-12-13	Устранение взаимной блокировки, внесённой 218f51584d5	12 → 10	5
52ac6cd2d0c	2018-12-13	Запрет преждевременного переиспользования удалённых страниц	12 → 9.4	5
c6ade7a8cd3	2018-12-13	Устранение взаимной блокировки при воспроизведении журнала	12 → 9.4	5

Продолжение на следующей странице

Продолжение таблицы А.1

Коммит	Дата	Изменение	Выпуск	Гл.
fe280694d0d	2019-03-05	Обход в физическом порядке страниц при сборке мусора	12	5
f2e403803fe	2019-03-10	Покрывающие атрибуты в обобщённом дереве поиска	12	3
7df159a620b	2019-03-22	Удаление опустевших страниц при сборке мусора	12	5
9eb5607e699	2019-07-24	Уточнение проверок удалённых страниц	13 → 12	5
6655a7299d8	2019-07-24	Полный 64-разрядный идентификатор транзакции для удалённой страницы	13 → 12	5
6754fe65a4c	2019-08-12	Пропуск нежурналируемых отношений при проверке на реплике	13 → 10	6
e5d8f359610	2019-09-08	Обработка Inf и NaN в очереди приоритетов поиска соседей	13 → все	3
39132b784ae	2020-08-08	Проверка соседних ссылок со сцеплением блокировок	14	6
16fa9b2b30a	2020-09-17	Построение обобщённого дерева поиска сортировкой	14	4
265ea567852	2020-10-01	Установка правых ссылок при сортированном построении	14	4
6f0bc5e1daf	2020-10-30	Проверка функции поддержки сортировки при создании индекса	14	4
756ab29124d	2021-01-13	Средства инспекции страниц обобщённого дерева поиска	14	7
09c1c6ab4bc	2021-04-05	Покрывающие атрибуты в дереве с разбиением пространства	14	3
3cd9c3b9219	2021-10-23	Учёт подготовленных транзакций при конкурентном создании индекса	15 → 9.6	5
f1ea98a7975	2022-02-07	Снижение перекрытия нелистовых ключей при сортированном построении	15	4
4ed8f0913bf	2023-11-29	64-разрядная адресация служебных журналов транзакций	17	5
b1fe8efdf17	2024-03-23	Нормализация индексных кортежей при проверке	17 → все	6
ab65dfb0fb2	2024-03-23	Поддержка разных размеров заголовка при нормализации	17 → все	6

Продолжение на следующей странице

Продолжение таблицы А.1

Коммит	Дата	Изменение	Выпуск	Гл.
53d3daa491b	2025-02-18	Специализация сортировки целочисленных массивов	18	4
c5c239e26e3	2025-03-21	Упреждающее чтение при сборке мусора в В-дереве	18	5
69273b818b1	2025-03-21	Упреждающее чтение при сборке мусора в обобщённом дереве поиска	18	5
e215166c9c8	2025-03-21	Упреждающее чтение при сборке мусора в дереве с разбиением пространства	18	5
d70b17636dd	2025-03-29	Выделение общих процедур проверки в отдельный модуль	18	6
14ffaese0fb	2025-03-29	Проверка структурных инвариантов обратного индекса	18	6
e4309f73f69	2025-04-03	Сортированное построение для классов операторов <code>btree_gist</code>	18	4
8ec97e78a77	2025-09-08	Коды ошибок при обнаружении повреждения данных	19	6

Итого 37 изменений в выпусках 10–19; часть из них дополнительно перенесена в выпуски вплоть до 9.4. Каждому изменению соответствует публично архивированное обсуждение, ссылки на которое приведены в списке использованных источников.

В ДОКУМЕНТЫ О ВНЕДРЕНИИ